PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

A Project Report
On

AI-Assisted 3D Assembly Design

Predicting Missing Components in CAD Assemblies using Graph Neural Networks

Submitted in fulfilment of the requirements for
Project Phases – 1, 2 and 3 (Consolidated Report)

Submitted by

Parthasarathy Perumal
SRN: PES2PGE24DS193

Under the guidance of
Prof. Sagarika Borah (Phase 1)
Prof. Gaurav Siwal (Phases 2 & 3)
Department of Computer Science and Engineering

May – August 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING

PROGRAM: M.TECH – DATA SCIENCE & ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the project report entitled **"AI-Assisted 3D Assembly Design: Predicting Missing Components in CAD Assemblies using Graph Neural Networks"** is a bonafide work carried out by **Parthasarathy Perumal (SRN: PES2PGE24DS193)** in fulfilment of Project Phases – 1, 2 and 3 in the Program of Study, Master of Technology in Data Science & Artificial Intelligence, under the rules and regulations of PES University, Bengaluru, during the period May 2026 – August 2026, under the guidance of Prof. Sagarika Borah (Phase 1) and Prof. Gaurav Siwal (Phases 2 and 3). It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work for this semester.

Signature with date	Signature with date	Signature with date
Internal Guide	**Chairperson**	**Dean of Faculty**
Prof. Gaurav Siwal		

Name and Signatures of the Examiners

1.

2.

DECLARATION

I hereby declare that Project Phases – 1, 2 and 3, entitled **"AI-Assisted 3D Assembly Design: Predicting Missing Components in CAD Assemblies using Graph Neural Networks"**, have been carried out by me under the guidance of Prof. Sagarika Borah (Phase 1) and Prof. Gaurav Siwal (Phases 2 and 3), Department of Computer Science and Engineering, and submitted in fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science & Artificial Intelligence of PES University, Bengaluru, during the academic period May – August 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

SRN: PES2PGE24DS193 Parthasarathy Perumal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Sagarika Borah, Department of Computer Science and Engineering, PES University, for her continuous guidance, feedback, and encouragement during Phase 1 of this project. Her suggestion during the First Review to strengthen the geometric grounding of the feature pipeline, and her published work on Octree-based spatial partitioning for polygon mesh analysis, directly shaped two of the key modules described in this report – the geometry-driven node feature enrichment and the open surface detection module.

I am equally grateful to Prof. Gaurav Siwal, Department of Computer Science and Engineering, PES University, for taking over as internal guide from Phase 2 onward and for his steady direction through the heterogeneous-encoder redesign, the next-component ranking work, and the Phase 3 shape-generation module – and in particular for pushing for the honest, root-cause-first treatment of the NodeRanker majority-class-collapse investigation documented in Chapter 6, rather than a quietly reworded metric.

I am also thankful to the project review panel and coordinators at PES University for their time and constructive comments across the zeroth, first, second, and third review sessions, and the Phase 2 guidance calls, all of which helped correct course at several points during the project. Finally, I extend my thanks to my family and friends for their patience and support through the course of this work.

ABSTRACT

Engineering CAD assemblies consist of multiple interconnected components whose correct selection is time-consuming and expertise-dependent. Modern CAD tools such as SolidWorks, CATIA, and Fusion 360 provide no contextual intelligence during assembly creation, forcing engineers to rely entirely on accumulated domain knowledge to choose and position components. This project addresses that gap in three phases: (1) missing-component **detection** via link prediction on a geometry-derived assembly graph, (2) next-component **ranking** to recommend what to add, and (3) missing-component **shape generation** to produce an actual 3D mesh for the recommended part – an end-to-end pipeline from a partial STEP assembly to a suggested, correctly-placed, correctly-shaped component.

Each CAD assembly is converted into an attributed graph using gmsh with the OpenCASCADE kernel: solid bodies become nodes and physical contacts between bodies become edges. The 22-dimensional node feature vector combines geometry-driven component typing, exact mesh surface area, affine-invariant shape descriptors, Shape Diameter Function statistics, and hole-count metadata; the 6-dimensional edge vector encodes mate type, contact weight, and joint type – both progressively de-hardcoded across the project, from placeholder constants in Phase 1 to fully geometry-derived values by Phase 2. Phase 1′s homogeneous 3-layer Graph Attention Network was replaced in Phase 2 by a **heterogeneous relation-aware encoder** (`RGATConv` with `TypedLinear` per-component-type projections, over a redesigned 8-class component taxonomy), feeding both a **LinkPredictor** head (Phase 1 detection) and a **NodeRanker** head (Phase 2 ranking, trained with a Bayesian Personalised Ranking loss over cosine similarity to learned type prototypes). Phase 3 adds a `ConditionalShapeVAE` – a 32^3 occupancy-grid convolutional variational autoencoder conditioned on the frozen encoder's context embedding – wrapped in a `HybridShapeGenerator` that prefers nearest-neighbour retrieval from a part bank and falls back to VAE synthesis only when no retrieval candidate fits well.

Across 38 documented training iterations (R1–R38) plus a final consolidated end-to-end retrain, spanning dataset scale-ups, an 8-class taxonomy migration, a homogeneous-to-heterogeneous encoder rewrite, and systematic feature/split/promotion-gate bug fixes, the final Phase 1 model – retrained on an expanded 238-model, 233-graph corpus across seven real-world assembly categories – reaches mean AUC-ROC 0.8235 ± 0.0219 and mean Average Precision 0.8114 ± 0.0279 under 5-fold cross-validation (best fold validation AUC 0.8607), a +0.136 AUC and +0.142 AP improvement over the immediately preceding serving checkpoint, and within reach of the Phase 1 targets (AUC ≥ 0.85, AP ≥ 0.82) for the first time in the project's history. The Phase 2 NodeRanker, retrained against the same final encoder, reaches Hit@1 0.4267, genuinely beating its majority-class baseline of 0.4133 for the first time after a documented three-step root-cause investigation into an earlier majority-class-collapse failure mode. The Phase 3 shape generator reaches voxel IoU 0.6277 and chamfer-proxy 0.0356, comfortably clearing its design targets (IoU ≥ 0.35, chamfer ≤ 0.08). The complete pipeline – detection, ranking, and shape generation – is verified end-to-end on real STEP files and deployed as an interactive Streamlit application with real-time 3D visualisation and

a Gemini-powered natural-language explanation agent (AIDA). The solution is implemented entirely in Python using PyTorch Geometric, without dependency on any proprietary CAD software API.

Table of Contents

List of Figures

List of Tables

1 Introduction

Computer-Aided Design (CAD) is the cornerstone of modern engineering, enabling the creation, modification, and optimisation of designs in a digital environment. CAD assemblies – collections of multiple interconnected mechanical components – are central to industries ranging from automotive to aerospace, consumer electronics, and industrial machinery. Assembling components in a CAD environment requires selecting the right parts, positioning them correctly, and defining the appropriate mate constraints (coincident, concentric, tangent, and so on) between them.

Despite decades of CAD tool evolution, the assembly process remains largely manual and expertise-dependent. Engineers rely on years of accumulated domain knowledge to identify which components belong in an assembly, to notice when parts are missing, and to determine an appropriate assembly sequence. This creates practical difficulties: junior engineers face a steep learning curve, different designers make inconsistent choices for equivalent subassemblies, and a partially defined assembly has no built-in mechanism to flag, recommend, or fill missing parts.

This project explores the use of **Graph Neural Networks (GNNs)** for intelligent assembly assistance, developed across three phases. By representing a CAD assembly as a graph – solid bodies as nodes, physical contacts between bodies as edges – the problem of detecting a missing component maps naturally onto **link prediction** (Phase 1): identifying an edge that should exist between two nodes but is currently absent. Once a gap is detected, recommending **what** component belongs there maps onto **ranking** (Phase 2): scoring candidate component types against the local graph context. Finally, producing an actual, placeable 3D mesh for that recommendation maps onto **conditional generation** (Phase 3): synthesising or retrieving a shape consistent with the target region's geometry.

1.1 Background

An assembly graph built directly from raw STEP geometry, without any proprietary CAD-vendor metadata, only has access to what can be derived purely from geometry: volumes, surface areas, bounding boxes, spatial positions, and detected physical contacts between solids. This geometry-only constraint was adopted deliberately so that the resulting pipeline works identically on STEP files exported from SolidWorks, CATIA, Fusion 360, or any other ISO 10303-compliant CAD system, rather than being tied to one vendor's proprietary assembly-tree format. This constraint was carried through unchanged from Phase 1 into Phases 2 and 3: neither the heterogeneous encoder, the ranking head, nor the shape generator depends on anything beyond STEP geometry and metadata computed from it.

1.2 Problem Statement

Modern CAD tools offer no contextual intelligence during assembly creation. Engineers rely entirely on domain knowledge to choose, position, and shape each component. The key pain points are:

1. **Knowledge barrier** – junior engineers lack the assembly patterns that experienced designers build up over years of practice.
2. **No automation** – existing CAD tools offer no intelligent part suggestion, ranking, or missing-component detection during assembly creation.
3. **Non-standardisation** – different designers make inconsistent choices for structurally equivalent subassemblies, creating quality and maintenance issues downstream.
4. **Incomplete assemblies** – a partially defined assembly has no built-in mechanism to detect, rank, or fill missing parts automatically.

Research gap: no existing system applies graph-based deep learning to detect, rank, and generate the shape of missing components from a partial CAD assembly graph without depending on a proprietary CAD API.

1.3 Objectives

1. **Graph construction pipeline** – build a robust pipeline that converts STEP files into attributed assembly graphs, using gmsh (OpenCASCADE) with geometry-enriched node and edge features.
2. **Missing component detection (Phase 1)** – train a graph-attention encoder with a Link Predictor head to detect missing connections in a partial assembly, targeting AUC-ROC ≥ 0.85 and Average Precision ≥ 0.82.
3. **Next-component ranking (Phase 2)** – train a heterogeneous, relation-aware encoder together with a NodeRanker head to rank candidate component types for a detected gap, targeting Hit@5 ≥ 0.70 and MRR ≥ 0.64.
4. **Missing-component shape generation (Phase 3)** – build a part-bank retrieval and conditional voxel-VAE pipeline that produces a plausible 3D mesh for the top-ranked missing component, targeting voxel IoU ≥ 0.35 and chamfer-proxy ≤ 0.08.
5. **Assembly completeness model** – build a template-based system that learns per-category component-type distributions and identifies which component types are missing from a partial assembly.
6. **Open surface detection** – implement a spatial analyser that localises the precise physical surfaces where a missing component should be attached.
7. **AI-powered explanation** – integrate a Gemini-based Skills AI agent that translates raw GNN scores into actionable engineering language.
8. **Interactive deployment** – ship the system as a Streamlit web application supporting upload, 3D visualisation, detection, ranking, generation, and explanation in a single workflow.

1.4 Scope of the Project

The project was planned and executed across three phases:

- **Phase 1 (May – July 2026, complete):** base replication – missing-component detection via a GAT-based link predictor, the assembly completeness model, open surface detection, the AIDA Skills AI agent, and Streamlit deployment. Evaluation metrics: AUC-ROC and Average Precision.

- **Phase 2 (July – August 2026, complete):** novelty and improvement – a heterogeneous, relation-aware encoder (`RGATConv` + `TypedLinear`) replacing the homogeneous GAT, an 8-class component-type taxonomy migration, and next-component ranking via a `NodeRanker` head trained with a BPR ranking loss. Evaluation metrics: Hit@K, MRR, NDCG@K.
- **Phase 3 (August 2026, complete):** missing-component shape generation – a part-bank retrieval system and a `ConditionalShapeVAE`, combined in a retrieval-first `HybridShapeGenerator`, producing an actual mesh for the top-ranked missing component. Evaluation metrics: voxel IoU and a chamfer-distance proxy.

The system operates on standard STEP/STP files (ISO 10303) and does not require any proprietary CAD software API, making it portable across CAD platforms. This consolidated report documents all three phases, from the project's first commit (13 May 2026) through the final end-to-end retrain (17 August 2026), ahead of the End-Semester examination on 21 August 2026.

1.5 Organisation of the Report

- **Chapter 2 – Literature Survey** reviews existing research in GNN-based geometric learning, CAD representation learning, ranking losses, generative shape modelling, and mesh spatial analysis.
- **Chapter 3 – System Requirements Specification** details hardware, software, functional, and non-functional requirements across all three phases.
- **Chapter 4 – Proposed Methodology** describes the system architecture, the graph construction pipeline, the Phase 1 detection model, the Phase 2 heterogeneous encoder and ranking head, the Phase 3 shape-generation pipeline, and the complementary analysis modules.
- **Chapter 5 – Implementation Details** covers the development environment, dataset processing across all phases, and evaluation approach.
- **Chapter 6 – Results and Discussion** presents results across 38 documented training iterations plus the final consolidated retrain, and reports Phase 1, 2, and 3 metrics against their targets.
- **Chapter 7 – Conclusion and Future Work** summarises the outcomes of all three phases and outlines remaining open work.

2 Literature Survey

This chapter reviews research relevant to the project, spanning graph neural networks, ranking losses, B-Rep generative models, CAD representation learning, generative shape modelling, and spatial mesh analysis, and identifies the gaps that this project sets out to address.

2.1 Graph Neural Networks for Link Prediction

Kipf & Welling (2017) introduced the Graph Convolutional Network (GCN), which applies spectral convolutions to graph-structured data by aggregating neighbour features through a normalised adjacency matrix. GCN performs well on node classification and link prediction benchmarks but treats all neighbours with equal importance, lacking a mechanism to weight the relative significance of different connections.

Veličković et al. (2018) proposed the Graph Attention Network (GAT), which introduces learnable attention coefficients into the neighbourhood-aggregation step. Multi-head attention allows the model to attend to several structural patterns simultaneously. GAT has been shown to outperform GCN on heterogeneous graphs where neighbour importance varies – a property directly relevant to assembly graphs, where different mate types (coincident, concentric, tangent) carry different structural significance. This project's Phase 1 encoder is a direct application of GAT; Phase 2 extends the same attention principle into a **relation-aware** setting (Section 4.4), where the joint-type edges themselves are treated as distinct relations rather than a single homogeneous edge type.

Hamilton, Ying & Leskovec (2017) introduced GraphSAGE, which learns node embeddings by sampling and aggregating features from a node's local neighbourhood, supporting inductive learning on nodes unseen during training – relevant to inference on assemblies not present in the training corpus. GraphSAGE was directly re-examined in this project as a benchmark encoder alongside the production RGAT encoder (Section 6.4); a topology-and-feature-only aggregator with zero edge features proved competitive at full cross-validation rigor, a result discussed in Chapter 6.

An anonymous 2024 study, "Can GNNs Learn Link Heuristics?", investigates whether GNNs can recover classical link-prediction heuristics such as common-neighbour count, the Jaccard coefficient, and the Adamic–Adar index directly from graph structure, and finds that explicit structural features can usefully complement learned representations.

2.2 Ranking Losses and Recommendation

Rendle et al. (2009) proposed **Bayesian Personalised Ranking (BPR)**, a pairwise ranking loss that directly optimises the probability that an observed (positive) item is ranked above an unobserved (negative) item, rather than treating recommendation as independent binary classification. BPR is the loss function adopted for this project's Phase 2 `NodeRanker` head (Section 4.5): each training example pushes the cosine similarity between a partial-assembly context vector and the true missing component type's learned prototype above the similarity to the other seven types, which is a direct instantiation of BPR's pairwise

formulation over a fixed candidate set of eight component-type prototypes rather than an open item catalogue.

2.3 B-Rep Generative Models and CAD Synthesis

Du et al. (2024), **BrepGen**, introduced a B-Rep generative diffusion model with structured latent geometry, generating boundary-representation solids by learning latent codes for faces, edges, and vertices and decoding them into valid CAD geometry. The work targets single-part generation rather than assembly analysis, but demonstrates that deep learning can capture the geometric structure of B-Rep models.

Jayaraman et al. (2024), **SolidGen**, proposed an autoregressive model for direct B-Rep synthesis and editing, generating CAD construction sequences step by step. The autoregressive framing is conceptually related to next-component prediction in assembly design, where each added component depends on the existing partial assembly – the direction implemented in this project's Phase 2 ranking head.

Wang et al. (2025), **CAD-GPT**, synthesises CAD construction sequences using spatial-reasoning-enhanced multimodal LLMs, combining language understanding with 3D spatial reasoning to generate models from natural-language descriptions. This highlights the potential of AI-assisted CAD workflows, but again at the individual-part rather than assembly level.

2.4 Generative Shape Modelling

Kingma & Welling (2014) introduced the **Variational Autoencoder (VAE)**, a generative model that learns a probabilistic latent space by jointly training an encoder (approximating the posterior over latent codes) and a decoder (reconstructing the input from a sampled latent code), optimised via a reconstruction term plus a KL-divergence regulariser against a prior. This project's Phase 3 `ConditionalShapeVAE` (Section 4.6) is a direct, conditional application of this framework to voxel-occupancy grids: the encoder/decoder operate over a 32^3 occupancy volume, and the latent code is additionally conditioned on the frozen Phase 1/2 encoder's context embedding, the target component type, and target bounding-box/scale information, so that the decoder generates a component shaped and sized to fit its specific gap rather than an unconditional sample from the class.

2.5 Heterogeneous Graph Learning

An anonymous 2023 paper, "Heterogeneous Graph Contrastive Learning", proposes contrastive learning for heterogeneous graphs with multiple node and edge types. This is directly relevant to CAD assemblies, where nodes represent different component types and edges represent different mate constraints. Phase 2 of this project builds on these ideas directly: the homogeneous GAT encoder from Phase 1 was replaced by a heterogeneous `RGATConv` + `TypedLinear` encoder (Section 4.4) that models component types and joint-relation types with distinct, learned parameters rather than a single shared parameter set.

2.6 Assembly Analysis and Spatial Reasoning

Jones et al. (2021), "Learning Bottom-up Assembly of Parametric CAD Joints", addresses the prediction of assembly joints between parametric CAD parts. This is the closest prior work to the objectives of this project, but relies on parametric CAD representations and vendor-specific joint annotations, whereas the approach adopted here works from geometry alone throughout all three phases.

Koch et al. (2019) released the **ABC Dataset**, a collection of over one million CAD models in STEP/B-Rep format with geometric metadata such as bounding boxes, volume, and surface area – a foundational resource for geometric deep learning on CAD models.

Mo et al. (2019) contributed **PartNet**, a benchmark of 573,585 3D parts across 26 object categories with fine-grained, instance-level, and hierarchical part annotations. PartNet's hierarchical annotation style informed the edge-feature construction approach used in this project.

Willis et al. (2021) released the **Fusion 360 Gallery Dataset**, containing 8,251 assemblies and 154K bodies extracted from the Autodesk Fusion 360 platform. This project integrated 643 assembly STEP files from the Fusion 360 Gallery during the Phase 1 corpus-scaling exploration (Section 6.3); the corpus used for the final, cross-phase retrain (Section 6.7) is instead a curated, deduplicated set of real-world mechanical assemblies described in Section 5.2.

2.7 Mesh Processing and Spatial Partitioning

Borah & Borah (2020) proposed a Prediction Error Expansion (PEE) based reversible polygon-mesh watermarking scheme that uses Octree spatial partitioning for regional tamper localisation. An Octree divides the mesh's bounding volume into independent spatial sub-blocks, and each block's vertices are authenticated separately so that tampering can be localised to a region rather than merely detected globally. This spatial-decomposition principle – localised, per-region decisions from a global structure – is directly adapted in this project's open surface detection module, where an Octree partitions free-surface centroids to localise where a missing component should be placed.

Ying et al. (2019), **GNNExplainer**, introduced a model-agnostic method for explaining GNN predictions by identifying the subgraph structures and node features most responsible for a given prediction. GNNExplainer is now integrated (Section 4.6) – wrapping the frozen encoder and task heads to attribute each missing-link and next-component prediction to specific message-passing edges and node features, wired live into the Streamlit UI. Its role in this project's root-cause style of reporting during Phases 2 and 3 was, before integration, substituted by manual, hypothesis-driven ablations (Sections 6.4-6.6); the two approaches are complementary rather than exclusive, and both remain part of the project's methodology.

2.8 Summary and Limitations of Existing Systems

The reviewed literature reveals five recurring limitations that this project's design directly responds to:

1. **Part-level focus** – most generative CAD models (BrepGen, SolidGen, CAD-GPT) operate at the level of an individual part, not at the assembly level where component interactions matter.
2. **Proprietary API dependency** – systems such as the Fusion 360 joint-prediction work rely on vendor-specific CAD APIs for constraint metadata, limiting portability across CAD platforms.
3. **No geometry-driven component typing** – existing approaches typically require a labelled component-type dataset, rather than inferring type directly from geometric signal.
4. **No spatial localisation of the gap, and no generated fix** – no reviewed system both identifies the precise physical surface where a missing component should be placed **and** produces an actual, fitted 3D shape for it; existing work stops at naming what type is missing.
5. **No engineering-language explanation** – predictions are typically presented as raw numerical scores rather than translated into an actionable recommendation.

3 System Requirements Specification

This chapter specifies the hardware and software environment, and the functional and non-functional requirements the system must satisfy across all three phases.

3.1 Introduction and Project Scope

The system accepts a STEP/STP assembly file as input, analyses it through a geometry-parsing pipeline, a trained heterogeneous encoder feeding a link predictor, a next-component ranker, and a shape generator, a template-matching module, and a spatial open-surface detector, and returns a structured, colour-coded analysis – including, where applicable, a generated 3D mesh for the top-ranked missing component – together with a natural-language explanation, rendered inside an interactive 3D viewer. Training a new model on a fresh corpus of STEP files, and running inference against an already-trained model, are both first-class, user-triggerable workflows in the same application.

3.2 Functional Requirements

ID	Requirement
FR1	Parse STEP/STP files (ISO 10303) and extract solid bodies, volumes, surface areas, bounding boxes, spatial positions, and surface-contact topology.
FR2	Convert a parsed assembly into an attributed PyG graph with geometry-derived node and edge features.
FR3	Train a heterogeneous, relation-aware encoder with a Link Predictor head using k -fold cross-validation, with hyperparameters configurable via a single YAML file.
FR4	Predict missing connections between assembly components and report the top-K candidates with confidence scores.
FR5	Identify the assembly category (e.g. Bench Vice, Gate Valve) from an uploaded STEP file, with a confidence score.
FR6	Compare an uploaded assembly against learned per-category templates to identify which component types are missing.
FR7	Detect and visualise unmated surfaces where a missing component should attach.
FR8	Generate a structured engineering-language explanation of the predictions using the Gemini-based Skills AI agent.
FR9	Render an uploaded assembly in an interactive 3D viewer with colour-coded highlights for each analysis category.
FR10	Support single-body uploads by inferring the component type geometrically, matching it to an assembly category, and reporting the components expected to complete that assembly.
FR11	Rank candidate component types for a detected gap using a frozen-encoder ranking head, and report the top-ranked type with its Hit@K / confidence context.

FR12	Generate an actual 3D mesh for the top-ranked missing fastener-type component, preferring part-bank retrieval and falling back to conditional VAE synthesis when no retrieval candidate fits well, and render it in the 3D viewer.

3.3 Non-Functional Requirements

ID	Requirement
NFR1	STEP parsing and inference shall complete within 60 seconds for assemblies with ≤ 20 bodies; the full detect-rank-generate pipeline shall complete within approximately 30 seconds on typical hardware.
NFR2	The system shall remain tractable for assemblies ranging from 2 to 448 bodies (the observed dataset range) via configurable graph-size filters.
NFR3	No dependency on a proprietary CAD API (SolidWorks, CATIA, Fusion 360) – standard STEP files only, across all three phases.
NFR4	The Streamlit interface shall provide upload, training, and inference workflows with visual feedback (progress indicators, streamed activity log, colour-coded analysis panels).
NFR5	All experiments shall use fixed random seeds, deterministic assembly-ID-based splits, hash-stable test carve-outs, and version-tracked configuration to ensure reproducibility.
NFR6	The AIDA agent shall degrade gracefully to an offline mode when no Gemini API key is configured, and single-body uploads shall trigger a geometry-only analysis path without requiring GNN inference.
NFR7	Model promotion from a new training run to the live serving checkpoint shall pass through an automatic, evidence-based gate (new run beats the incumbent on the tracked metric) except where a documented, deliberate manual override is recorded, so that a regression cannot silently reach production.

3.4 Hardware Requirements

Component	Specification
Processor	Apple M-series (ARM64) chip, or an equivalent x86_64 CPU with $\geq$ 8 cores
Memory (RAM)	$\geq$ 16 GB (32 GB recommended for large assemblies and Phase 2/3 retraining)
Storage	$\geq$ 50 GB SSD for dataset, checkpoints, part bank, and processed graph cache
GPU	Apple Metal Performance Shaders (MPS) for PyTorch acceleration; NVIDIA CUDA optional on x86
Display	$\geq 1920 \times 1080$ for the interactive 3D viewer

Table 1: Hardware Requirements

3.5 Software Requirements

Software	Version / Role
Operating System	macOS 13+ (ARM64) / Ubuntu 22.04+ / Windows 11
Python	3.10+ (developed on 3.12)
Package Manager	`uv`
PyTorch	2.x, MPS or CUDA backend
PyTorch Geometric	2.x – `RGATConv`, `GATConv`, `BatchNorm`, `RandomLinkSplit`, `DataLoader`
gmsh	4.x with OpenCASCADE kernel
trimesh	Latest – exact surface area, SDF ray-casting, voxelisation for the part bank
Streamlit	1.x – interactive web application
Plotly	5.x – 3D mesh visualisation (`Mesh3d`)
PyVista	Latest – offscreen STL loading
scikit-learn	Latest – AUC-ROC, Average Precision, `KFold`
Google Generative AI SDK	`google-generativeai` – Gemini API access

Table 2: Software Requirements

3.6 Use Case Overview

The primary use cases are: (1) an engineer uploads a STEP file, and the system parses it, constructs the graph, runs detection, ranking, and shape generation, and displays the analysis; (2) an engineer triggers training, and the system scans the configured source directory, builds the dataset, trains the relevant model(s), and reports metrics; (3) an engineer reviews the analysis panel together with the colour-coded 3D overlay, including any generated component mesh; (4) an engineer reads the AIDA explanation for a structured, natural-language summary of the same findings.

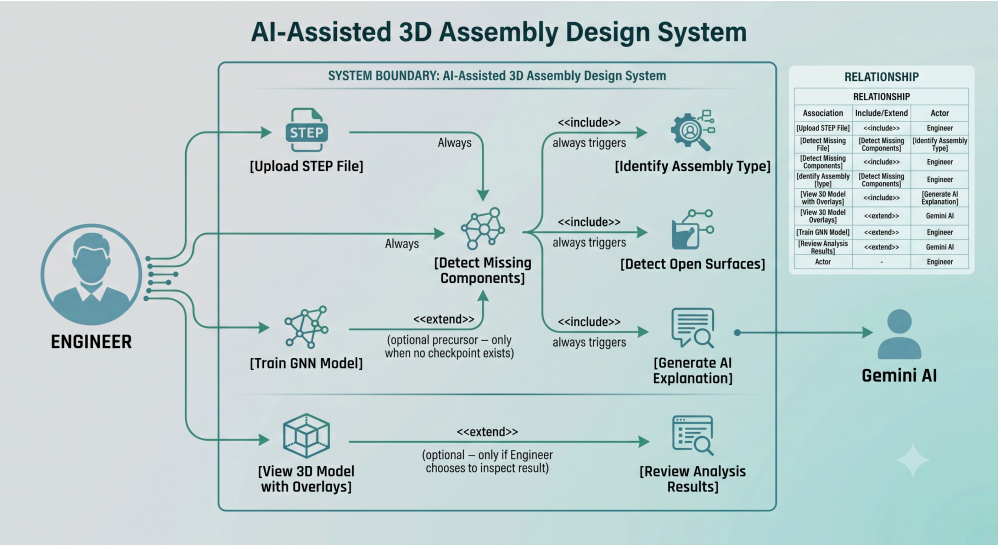

Figure 1: Use case diagram for the AI-Assisted 3D Assembly Design system.

4 Proposed Methodology

This chapter explains the system architecture, the graph construction pipeline, the Phase 1 detection model, the Phase 2 heterogeneous encoder and ranking head, the Phase 3 shape-generation pipeline, and the complementary analysis modules that together form the system.

4.1 System Architecture

The system follows a modular pipeline: (1) STEP parsing and graph construction, (2) a shared heterogeneous encoder producing node embeddings, (3) three task heads consuming those embeddings – a Link Predictor (Phase 1 detection), a NodeRanker (Phase 2 ranking), and a shape generator (Phase 3 generation) – (4) assembly-completeness and open-surface analysis running alongside the encoder, and (5) AI-powered explanation (AIDA) together with post-hoc model interpretability (GNNExplainer) and visualisation.

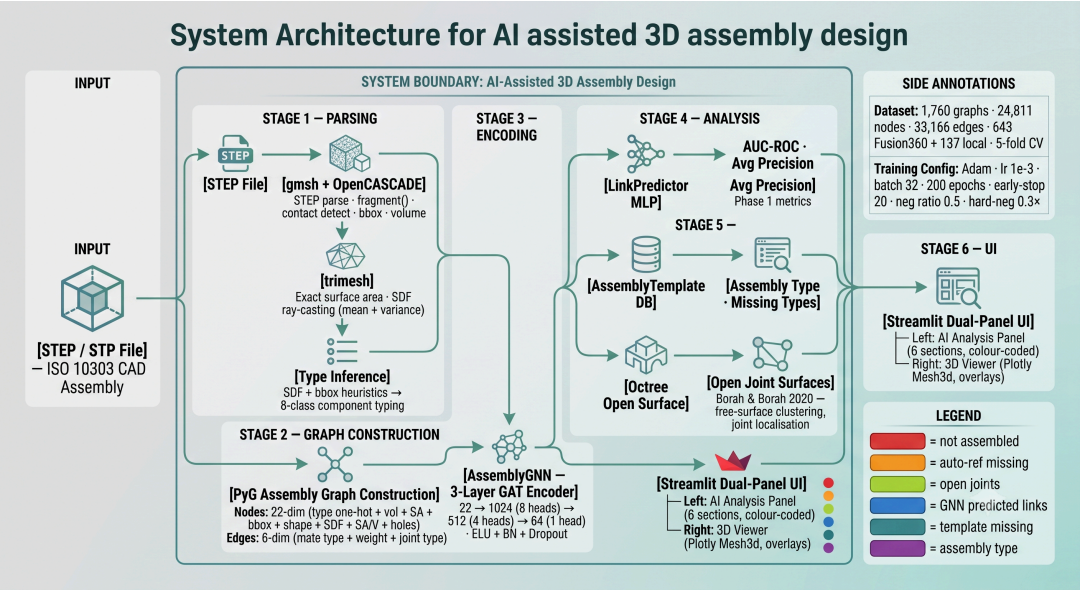

Figure 2: System architecture – from STEP file input, through graph construction and heterogeneous encoding, to the three task heads, the complementary analysis modules, and the Streamlit front end.

At a high level:

```
STEP file --> gmsh (OpenCASCADE) --> PyG assembly graph
  Nodes (22-dim): type one-hot(8, 8-class taxonomy) + log1p-vol + log1p-SA
    + bbox dx/dy/dz + affine-invariant shape(5)
    + SDF mean + SDF var + SA/V + log1p-holes
  Edges (6-dim): geometry-derived mate-type + real contact-area weight
    + joint-type one-hot(4)
  Node spatial position (pos) stored alongside, consumed by LinkPredictor

Assembly graph --> AssemblyGNN: heterogeneous 3-layer RGATConv + TypedLinear
  (22 -> 128 -> 64, [8,4,1]-head relational attention over 4 joint types)
  --> node embeddings (64-dim)
     |
     |--> LinkPredictor (MLP + relative-distance feature)
     |      --> binary edge scores --> missing component DETECTION (AUC-ROC, AP)
```

```
     |
     |--> NodeRanker (cosine sim. vs 8 type prototypes, learnable temperature)
     |      --> BPR loss --> next-component RANKING (Hit@K, MRR, NDCG@K)
     |
     '--> HybridShapeGenerator
            PartBank retrieval (nearest-neighbour + bbox refit) [preferred]
            ConditionalShapeVAE (32^3 occupancy, 128-dim latent, 77-dim cond.)
[fallback]
            --> missing component 3D mesh GENERATION (voxel IoU, chamfer)

Parallel:  AssemblyTemplateDB --> assembly type + missing component types
           OctreeNode          --> open surface joints

Skills AI: Gemini (gemini-2.0-flash) --> AIDA structured explanation
Interpretability: GNNExplainer (frozen encoder + heads) --> per-prediction edge/
feature attribution
All outputs --> Streamlit dual-panel UI (analysis panel + 3D viewer)
```

4.2 Graph Construction Pipeline

The graph construction pipeline (`back_end/dataset.py`) converts each STEP file into an attributed PyG `Data` object using a three-tool approach.

4.2.1 Step 1 – STEP Parsing with gmsh and OpenCASCADE

1. Load the STEP file and synchronise the OpenCASCADE kernel.
2. Perform **Boolean fragmentation** via `gmsh.model.occ.fragment()`. By default, STEP bodies are independent solids with no shared topology; `fragment()` performs a Boolean intersection of all volumes, forcing physically touching bodies to share the exact same boundary surface tags – without this step no shared surfaces, and therefore no edges, could be detected.
3. For each solid body, extract volume (`getMass(3, tag)`), bounding box (`getBoundingBox()`), centroid position, and the set of boundary surface tags (`getBoundary()`).
4. Declare an edge between two bodies whenever their boundary surface tag sets intersect – this is the operational definition of "physical contact" used throughout the project.
5. A bounded, wall-clock-deadline polling loop replaces an earlier unconditional `p.join()` after `p.kill()` in the timeout wrapper: an OpenCASCADE/gmsh C-extension call stuck in an uninterruptible state does not guarantee instant death on `SIGKILL` on macOS, and the earlier approach could block a parse for minutes.

4.2.2 Step 2 – Geometry Enrichment with trimesh

1. Export each body as an STL mesh via gmsh.
2. Compute the **exact surface area** using `trimesh.Trimesh.area`. This replaced the bounding-box approximation $2(dx \cdot dy + dy \cdot dz + dz \cdot dx)$ used in the earliest version of the pipeline, which was identified as the single biggest accuracy flaw in the original 13-dimensional feature vector.
3. Compute **Shape Diameter Function (SDF)** statistics via trimesh inward ray-casting: sample N surface points, cast a ray inward along the surface normal at each, record

the first-hit distance (the local material thickness at that point), and aggregate across all sampled points into an SDF mean (average thickness) and SDF variance (shape complexity).

4. A meshing-resilience fix (Phase 3) ensures that a single surface gmsh's mesher cannot triangulate ("Impossible to mesh periodic surface") aborts extraction of only that surface rather than the whole file – the same class of fix `dataset.py`'s STEP parser already had, later ported into `part_bank.py`.

4.2.3 Step 3 – Component Type Inference

SDF statistics, bounding-box ratios, and (from the Phase 2 taxonomy migration onward) multi-signal voting drive a geometry-only classification of each body into one of eight component-type classes.

Index	Class (8-class taxonomy, Phase 2 onward)	Discriminating signal
0	`long_shaft`	Strong elongation ($> 5\times$), no through-hole
1	`short_shaft`	Elongated but below the long-shaft threshold, no through-hole
2	`thick_plate`	Flat (flatness ratio $< 30\%$), no through-hole, above the thin-plate thickness cutoff
3	`thin_plate`	Flat and below the thin-plate thickness cutoff ($< 8\%$ of max extent)
4	`bolt`	Elongated (2–$8\times$) with a through-hole vote plus a distinct head signal (cylinder-fill / COM-offset / face-count vote)
5	`washer`	Flat, near-planar, high through-hole vote confidence
6	`nut`	Compact (elongation $< 1.8\times$), through-hole present, not flat
7	`body`	Generic fallback – none of the above vote strongly

Table 3: Geometry-driven component-type classification rules, current 8-class taxonomy (migrated from the original body/fastener/bearing/shaft/plate/housing/gear/other scheme – see Section 6.3).

The taxonomy migration was triggered by a real bug: the original taxonomy's `bearing` class, present for only one example in the whole corpus, polluted a category's expected-components template and produced a spurious "1 bearing missing" report in the UI. `dataset.py` is now the single source of truth for the component-type list and classifier; `part_bank.py`, `app.py`, and the audit tooling all import from it rather than duplicating the rules. The classifier's cutoffs

(`_TYPE_THRESHOLDS`) were carried over verbatim from a pre-migration candidate scheme and have not been independently re-tuned against ground truth since the migration – a corpus-wide audit found 39.5% of bodies show vote conflicts between signals on the original 193-assembly corpus, rising to 46.8% after the corpus was expanded to 238 models (Section 5.2) – an acknowledged limitation discussed further in Chapter 7.

4.2.4 Node Feature Vector

Dim	Feature	Source
0–7	Component type one-hot (8 classes, current taxonomy)	Multi-signal geometry voting
8	$\log(1 + \text{volume})$, clipped	`gmsh.occ.getMass()`
9	$\log(1 + \text{surface area})$, clipped	`trimesh.Trimesh.area`
10–12	Bbox $\Delta x, \Delta y, \Delta z$ / bbox_max	`getBoundingBox()`
13	Elongation (longest / mid axis)	Sorted bbox axes
14	Flatness (min / max axis)	Sorted bbox axes
15–16	Aspect x/y, aspect y/z, normalised	`getBoundingBox()`
17	Sphericity: $\pi^{\frac{1}{3}} \frac{(6V)^{\frac{2}{3}}}{\text{SA}}$	gmsh + trimesh
18–19	SDF mean, SDF variance (normalised)	Inward ray-casting
20	SA/V ratio (normalised)	trimesh + gmsh
21	$\log(1 + n_{\text{holes}})$	Geometry-derived hole count – fixed to a real computation at run R35, see Section 6.4 (was JSON metadata pass-through in Phase 1)

Table 4: The 22-dimensional node feature vector, current state (Phase 2/3). Structurally unchanged from Phase 1′s schema (Table 4.4 of the original Phase 1 chapter) but with dims [0-7] and [21] now geometry-derived rather than metadata- or hardcode-dependent.

4.2.5 Edge Feature Vector

Dim	Feature	Description
0	Mate type	**Geometry-derived from run R35 onward** (was hardcoded 0.0/1.0 in Phase 1): from the shared contact surface(s)‘ geometric type – cylindrical face(s) → concentric, planar → coincident, otherwise → other
1	Weight	**Real contact-area ratio from run R36 onward** (was hardcoded 1.0 in Phase 1 and R35): shared contact-surface area ÷ the smaller body's total surface area, clipped $[0, 1]$
2–5	Joint-type one-hot	rigid, revolute, slider, cylindrical – **geometry-derived from run R35 onward**: cylinder-only shared face → revolute; cylinder + plane → cylindrical; plane-only → rigid; anything else → slider

Table 5: The 6-dimensional edge feature vector, current state (Phase 2/3).

The Phase 1 report flagged, as an acknowledged limitation, that mate type for every detected contact was hardcoded to 0.0 (coincident) because the true constraint type is not recoverable from raw STEP boundary geometry alone. Runs R35 and R36 resolved this by deriving mate type, edge weight, and joint type directly from the **geometric type** of the shared contact surface(s) rather than from unavailable CAD constraint metadata – closing this limitation without introducing the false CAD-API dependency the project was designed to avoid. In addition, from run R36 onward each node's spatial centroid (`pos`) is stored on the graph and consumed directly by the `LinkPredictor` as a relative-distance feature (Section 4.3), giving the model its first genuine geometric-proximity signal.

4.3 Phase 1 – Missing Component Detection

4.3.1 LinkPredictor – MLP Task Head

The Link Predictor scores a candidate edge (u, v) by concatenating the two node embeddings (and, from run R36 onward, a relative-distance feature derived from `pos`) and passing the result through a small MLP:

$$\text{score}(u, v) = \text{MLP}(\boldsymbol{z}_u \parallel \boldsymbol{z}_v \parallel d(u, v)) \in \mathbb{R}, \quad \boldsymbol{z}_u, \boldsymbol{z}_v \in \mathbb{R}^{64}$$

The MLP's first layer grew from `in_dim`$\times 2 \rightarrow$ `hidden` to accommodate the additional distance feature at R36 – a change that intentionally breaks strict-loading of any pre-R36 checkpoint, held back until the retrain that made it meaningful to avoid a window where the live application could not load its own model.

4.3.2 Training Procedure

Parameter	Value
Optimizer	Adam (lr $= 10^{-3}$, weight_decay $= 5 \times 10^{-4}$)
LR schedule	ReduceLROnPlateau (factor = 0.5, patience = 8)
Early stopping	Patience = 20 epochs
Max epochs	200
Batch size	16 graphs (reduced from 32 at run R31 – MPS out-of-memory mitigation)
Negative sampling ratio	1.0 per positive edge (raised from 0.5 at run R37, together with a `random_ap` chance-baseline diagnostic)
`disjoint_train_ratio`	0.25 (introduced at run R35 – without it, train supervision edges leaked into message passing)
Dropout	0.3 (found via a 2-fold capacity-ablation screen at run R38, confirmed at full 5-fold rigor)
Cross-validation	5-fold, category-stratified from run R35 onward (guarantees every category, down to the smallest, is represented in every fold)
Loss function	BCE with logits + hard-negative term

Table 6: Training hyperparameters, current state (`back_end/config.yaml`).

Hard negative sampling. For each positive-edge source node, the model locates the most similar non-neighbour node by cosine similarity of embeddings, and adds it as a hard negative with a $0.3\times$ weight in the loss. This discourages the model from relying purely on embedding similarity and forces it to discriminate genuinely connected nodes from merely similar ones.

Checkpoint selection and promotion. From run R35 onward, checkpoint selection compares against a 3-epoch smoothed validation AUC rather than the raw per-epoch value, reducing sensitivity to single-epoch noise. Promotion to `best_serving.pt` is gated automatically – a new run must beat the incumbent on the tracked metric – except for one documented, deliberate manual override (run R33, Section 6.3) where the encoder's node-feature type-slots changed under a taxonomy migration and the automatic AUC comparison would not have been an apples-to-apples one.

4.4 Phase 2 – Heterogeneous Encoder and Next-Component Ranking

4.4.1 Heterogeneous Relation-Aware Encoder (`RGATConv` + `TypedLinear`)

The homogeneous 3-layer GAT encoder from Phase 1 was replaced, starting at run R30, by a **heterogeneous** encoder that models joint-relation types and component types with distinct, learned parameters rather than a single shared set:

Layer	Input Dim	Output Dim	Heads	Notes
RGAT 1	22	$128 \times 8 = 1024$	8	Edge features injected; relational attention over 4 joint-type relations; `TypedLinear` per-component-type input projection
RGAT 2	1024	$128 \times 4 = 512$	4	Relation types on all 3 layers
RGAT 3	512	64	1	Single head, no concatenation

Table 7: `AssemblyGNN` heterogeneous encoder architecture, Phase 2 onward. Parameter count grew from 602K (homogeneous GAT) to 2.44M with the R30 heterogeneous rewrite.

Each layer applies relation-aware attention, computing separate attention coefficients and transformation weights per joint-type relation $r \in \{\text{rigid, revolute, slider, cylindrical}\}$:

$$\boldsymbol{h}_i^{(l+1)} = \sigma\left(\sum_r \sum_{j \in \mathcal{N}_r(i)} \alpha_{ij}^{r,(l)} \boldsymbol{W}_r^{(l)} \boldsymbol{h}_j^{(l)}\right)$$

`TypedLinear` additionally applies a per-component-type input/output projection (8 types, matching the taxonomy in Table 4.1), so that, for example, a bolt and a plate route through different learned linear transforms before the shared relational-attention layers. Each layer is followed by an ELU activation, batch normalisation, and dropout ($p = 0.3$, Table 4.3).

An `encoder_type` switch (rgat / gatv2 / sage / gin, default rgat) was added at run R38 to benchmark the aggregation mechanism in isolation, holding per-stage width constant across all four. A 2-fold screen suggested GraphSAGE (topology + node features, zero edge features) clearly outperformed RGAT (0.6882 vs 0.6358 mean AUC), but a full 5-fold validation at production rigor essentially tied them (SAGE 0.6717 vs RGAT 0.6765 AUC; SAGE ahead on AP-lift by +0.024) – concrete evidence that a lean, reduced-fold protocol is a first-pass filter only, not a final architectural call. RGAT remained the production encoder; this benchmark also surfaced a near-miss where the 2-fold SAGE run's numbers legitimately beat the incumbent on both metrics and auto-promoted itself before being caught, leading to a `--no-promote` flag so exploratory benchmark runs can never touch the serving checkpoint regardless of outcome.

4.4.2 Next-Component Ranking – NodeRanker

`NodeRanker` shares the frozen Phase 1/2 encoder and adds a ranking head over the fixed set of 8 component-type prototypes:

$$\text{score}(c, t) = \text{logit\_scale} \cdot \cos(\text{proj}(\boldsymbol{c}), \boldsymbol{p}_t), \quad t \in \{1, ..., 8\}$$

where $\boldsymbol{c}$ is a mean-pooled context vector from the partial assembly graph (computed via leave-one-node-out sampling: a real node is removed from a training graph, the remainder is encoded through the frozen encoder, and the result is mean-pooled), $\boldsymbol{p}_t$ is a learned prototype embedding for component type t, and `logit_scale` is a CLIP-style learnable temperature scalar introduced at run R36. The model is trained with the **BPR** loss (Rendle et al., 2009; Section 2.2), which pushes the true removed node's type-prototype score above the other seven types' scores for each leave-one-out sample. `train_ranker.py` trains only the ranker projection against a frozen encoder, guarded against encoder staleness, and must be retrained whenever `best_serving.pt` is re-promoted so the ranker's cosine-similarity space stays synchronised with the encoder it is built on.

Root-cause investigation of an early majority-class-collapse failure. Early NodeRanker runs (R28-era through R34, and the initially-trained R36 checkpoint) all landed Hit@1 at or almost exactly at the majority-class baseline, despite the underlying encoder improving substantially across the same runs (mean AUC rising from around 0.55 to 0.6277 at R36) – indicating the bottleneck was in NodeRanker's own head, not the features feeding it. The investigation proceeded in three diagnostic steps rather than by guessing:

1. **Ruled out encoder quality** – Hit@1 tracked the baseline across every encoder generation tested, including the strongest one, ruling out "the encoder just isn't good enough" as the explanation.
2. **Ruled out prototype collapse** – a read-only diagnostic computed the 8×8 cosine-similarity matrix between the 8 learned type-prototype embeddings; they were reasonably separated (mean off-diagonal similarity ≈ -0.05, roughly orthogonal, maximum 0.34), ruling out "the prototypes have collapsed into each other".
3. **Confirmed majority-class collapse directly** – per-class Hit@1 and predicted-vs-true class-distribution logging showed the model predicting `body` for 94.3% of all test samples (against a true frequency of 42.9%), with Hit@1 near zero for every other type – functionally a majority-class classifier for 7 of 8 types, not a partial ranking failure.

Class-balanced BPR loss reweighting (inverse-frequency, capped at $5\times$) was tried as a fix: it reduced `body`'s predicted share from 94.3% to 88.6% and lifted `nut`'s Hit@1 from 0.000 to 0.143 – a real but weak effect – while aggregate Hit@1 ticked down slightly (0.4286 $\rightarrow$ 0.4190) and 5 of 7 non-body classes remained at exactly 0.000 Hit@1. Loss reweighting alone was concluded not to fix the underlying issue. The refined, not-yet-implemented hypothesis is that the **uniform mean-pool** context vector is the true bottleneck: with `body` at roughly 40% of most graphs, a uniform mean-pool over every surviving node plausibly barely shifts based on which specific node was removed, so the context vector mostly encodes "this graph is body-heavy" regardless of the true answer – an attention-weighted or removed-node-proximity-weighted pooling is the likely next step, held back pending design discussion rather than implemented unprompted (Section 7.2). Section 6.5 reports how this played out through R33/R34/R36 and into the final consolidated retrain, where Hit@1 genuinely beat the baseline for the first time.

4.5 Phase 3 – Missing-Component Shape Generation

Given a predicted-missing component type (from NodeRanker) and a target open-joint region (from the Octree open-surface detector), Phase 3 produces an actual 3D mesh. **Generation** is scoped to fasteners (bolt/washer/nut) only; detection and ranking are unaffected and still cover all 8 types – non-fastener misses continue to surface in the text-only expected-components panel without a generated mesh.

4.5.1 Part Bank (`part_bank.py`)

The part bank is a library of real, canonicalised part geometries extracted from the training corpus, used for retrieval-first generation. For each STEP file, every body is exported, canonicalised (rotated to a consistent reference frame), voxelised at 32^3 resolution, and indexed by component type and normalised scale, alongside its source mesh for direct retrieval. A **quality gate** introduced late in Phase 3 rejects bodies whose mesh is not watertight or whose volume is near zero (using |volume| rather than signed volume, a bugfix following the original gate's introduction) – both symptoms of a bad boolean-fragmentation or meshing result that would otherwise poison the bank with garbage geometry. The final part bank, rebuilt against the expanded 238-model training corpus (Section 5.2), contains 5,363 parts extracted from 237 assemblies, up from 3,843 parts across 193 assemblies before the corpus expansion.

4.5.2 `ConditionalShapeVAE`

The shape generator itself is a **conditional variational autoencoder** (Kingma & Welling, 2014; Section 2.3) operating over 32^3 binary occupancy grids:

- **Encoder/decoder**: a 3D-CNN over the occupancy grid, with a 128-dimensional latent code.
- **Condition vector (77-dim)**: the 64-dimensional frozen-encoder context embedding (mean-pooled, as in NodeRanker) concatenated with an 8-dimensional component-type one-hot, a 3-dimensional target bounding-box, and a 2-dimensional neighbour-scale signal ($64 + 8 + 3 + 2 = 77$) – so the decoder generates a shape sized and typed to fit the specific detected gap, not an unconditional class sample.
- **Loss**: binary cross-entropy (occupancy reconstruction) + soft-Dice (shape-overlap term) + KL-divergence (latent regularisation against the prior), with per-term loss logging added so a regression can be attributed to a specific term rather than one opaque scalar.
- **Orientation fix**: `rotate_to_target_axis()` originally compared only the shortest-vs-longest bounding-box extent to decide how to orient a generated part, which cannot distinguish a bolt's rod shape ($[5, 5, 20]$) from a washer's disk shape ($[5, 20, 20]$) when both hit the same extremes ratio – bolts were aligning their **short** (diameter) axis to the joint instead of their long (shaft) axis. Using the middle extent as well resolves the ambiguity.

4.5.3 `ShapeRetriever` and `HybridShapeGenerator`

`ShapeRetriever` performs nearest-neighbour search against the part bank (matched on component type and normalised scale) followed by a bounding-box refit of the retrieved mesh to the target gap. `HybridShapeGenerator` prefers retrieval whenever the retrieved candidate's

fit score clears `retrieval_tau` (0.6 for fasteners), and falls back to `ConditionalShapeVAE` synthesis only when no retrieval candidate fits well – reflecting the practical intuition that a real, manufacturable part geometry is preferable to a synthesised one whenever a good match exists.

4.5.4 Training & Evaluation (`train_shape_gen.py`)

Training uses the same leave-one-node-out sampling pattern as NodeRanker, against the frozen encoder, for 60 epochs with continuous-angle rotation and scale/occupancy-jitter augmentation. Two metrics are tracked: **voxel IoU** (deterministic, full-grid) and a **chamfer-distance proxy** over occupied-voxel centroid clouds, computed with a fixed-seed subsampling generator after re-evaluating the same checkpoint was found to give different chamfer values on different calls once a shape exceeded 300 occupied voxels ($\approx 110\%$ coefficient of variation from sampling noise alone). Checkpoint selection uses a composite score, IoU $-$ $2.0 \times$ chamfer, rather than IoU alone, after the earlier IoU-only rule was observed passing over a near-IoU-tied epoch with roughly 20% better chamfer purely because IoU ticked up slightly on a different epoch. A `ReduceLROnPlateau` schedule tracks the same composite score. An occupancy-jitter fix restricts augmentation noise to a `binary_dilation`-widened band around the occupied region, since a fastener occupies only 1-5% of the 32^3 grid and unrestricted jitter injected almost pure background noise. A retrieval-gated test metric filters test samples to the harder subset where the part bank's own retrieval fit score falls below threshold – i.e. the cases that would actually reach the VAE in production – though on the current test set, 0 of the evaluated samples fell below that threshold, meaning retrieval alone already covers the current test distribution at this threshold (Section 6.6).

4.6 Complementary Analysis Modules

4.6.1 Assembly Completeness Model (`AssemblyTemplateDB`)

`AssemblyTemplateDB` learns the expected component-type composition of each assembly category. During the **build phase**, run automatically after every training pass, it loads the processed graphs and their source paths, groups them by top-level source folder, computes the median count of each component type across all assemblies in that category, and caches the result to `assembly_templates.json`. A category-resolution bugfix (`_category_from_path()`) corrected an earlier assumption of fixed one-level nesting that had collapsed all seven real categories into a single blended template; a real Bench Vice upload's template-match confidence rose from 10.4% to 96.2% after the fix and a template rebuild.

At inference time, the match score for a candidate category is

$$\text{score} = \frac{\sum_{t \in \text{ types}} \min(\text{present}(t), \text{expected}(t))}{\max(\sum \text{present}, \sum \text{expected})}$$

with a 25% bonus applied when every present component type fits inside the template (that is, no unexpected component types appear). The minimum confidence threshold for a match is 0.10. Given a matched template, the gap between the template's expected counts and the counts actually present in the upload gives the list of missing component types – this

mechanism works even for a single-body upload, where the single body's inferred type is matched against every known template. The number of missing fasteners actually **generated** in Phase 3 is now driven directly by detected hole geometry and the part bank's shape-fit score rather than being hard-capped by this per-category median count, after the median count was found to under-generate for assemblies with many more real empty holes than the category median.

4.6.2 Open Surface Detection (Octree-based)

This module is adapted from the Octree spatial-partitioning concept described in Borah & Borah (2020), reused here for a structurally different purpose.

Concept	PEE Watermarking (2020)	This Project
Spatial partition	Octree of mesh vertices	Octree of free-surface centroids
Unit of analysis	Vertex block	Surface cluster
Per-block decision	Watermark valid / tampered	Surface mated / open joint
Localisation goal	Which region was modified?	Where is a component missing?

Table 8: Conceptual mapping from PEE watermarking's Octree partitioning to this project's open surface detector.

The pipeline: after `fragment()`, surfaces belonging to exactly one body ("free" surfaces) are identified; surfaces whose area is below 4% of the parent body's total surface area are discarded as fillets, chamfers, or noise; the remaining candidate surface centroids are inserted into an Octree of maximum depth 3 (up to $8^3 = 512$ leaves); from each non-empty leaf, the surface with the highest area ratio is selected as the representative open joint for that region; and a coarse triangle mesh is extracted for each flagged surface, for direct rendering in the 3D viewer. A later per-hole refinement adds a parallel candidate path using gmsh's cylindrical-face type query plus a diameter/depth plausibility filter and a pattern-repetition check (a real bolt-hole pattern repeats – two or more same-diameter holes on a body; a one-off functional bore does not, and is correctly excluded), fixing a case where 12 real empty bolt holes on a test assembly had been collapsed into a single whole-body region.

4.6.3 GNNExplainer – Post-hoc Interpretability

`back_end/explainer.py` wraps the frozen, already-trained encoder and task heads in a small adapter module bespoke to each explanation target – a specific candidate edge (u, v) for `LinkPredictor`, or a specific candidate component type for `NodeRanker` – reducing each to a single scalar score, and explains that scalar using PyTorch Geometric's `Explainer/GNNExplainer` in its "graph regression" mode (`explanation_type="model"`, `task_level="graph"`, `mode="regression"`), so the explanation stays faithful to the model's own raw score under feature/edge masking rather than being fit against a ground-truth label – a deliberate choice, since PyG's edge/node task-level batching conventions do not map cleanly onto this project's single-graph, single-candidate inference calls. Output is a ranked

list of the message-passing edges and node features that most influenced the given prediction, mapped back to real component types and feature names rather than raw tensor indices.

The module was verified end-to-end against both the synthetic demo assembly and a real STEP-derived graph (`C_Clamps_01`), on both the Phase 1 `LinkPredictor` and Phase 2 `NodeRanker` heads, using the final `Ph2-final-design` checkpoints. It is wired into the Streamlit front end (`app.py`): because the front end runs inference in a subprocess rather than calling `api.py` over HTTP (to avoid a `gmsh`/PyTorch signal-handler conflict, Section 5.1), the explainer is invoked inside that subprocess script, computing a 100-step GNNExplainer attribution for the single top missing-link prediction and the single top next-component pick, each guarded so that an explainer failure degrades to no explanation rather than breaking the inference pipeline. Two new collapsible panel sections, "Why This Missing Link?" and "Why This Next Component?", render the result in the same visual style as the existing prediction panels. It is skipped for single-body uploads, since a one-node, zero-edge graph has no message-passing structure for GNNExplainer to attribute across.

4.7 Skills AI – AIDA (Gemini-Powered Agent)

The `AssemblySkillsAgent` loads a domain skills profile (`engineering_3d_assembly.yaml`) spanning six skill areas: mechanical engineering, 3D modelling, 3D assembly design, parts identification, GNN-score interpretation, and assembly sequencing. At inference time AIDA receives the encoder's predicted missing links, the NodeRanker's top-ranked component type, the per-node degree information, the shape-generation result, and the open-surface analysis results, and produces a structured explanation covering: components not assembled or not properly mated, predicted missing links, the ranked recommendation for what to add, open assembly joints detected, and an overall recommendation. The agent degrades gracefully to an offline mode when no Gemini API key is configured, so the rest of the pipeline remains usable without it.

4.8 End-to-End Workflow

1. **Data preparation:** collect STEP files, apply quality filters (deduplication, zero-contact pre-filter, size limits, parsability audit), parse with gmsh, and build the cached PyG dataset and part bank.
2. **Training:** run 5-fold category-stratified cross-validation of the heterogeneous encoder + LinkPredictor with BCE loss and hard negatives; retrain NodeRanker (frozen encoder, BPR loss) and the shape generator (frozen encoder, BCE+Dice+KL loss) whenever the encoder is re-promoted; rebuild the template database and part bank.
3. **Inference:** upload a STEP file, parse it into a graph, run detection, ranking, template matching, and open-surface detection, generate a mesh for the top-ranked missing fastener if applicable, compute a GNNExplainer attribution for the top missing-link and next-component predictions, generate the AIDA explanation, and render the colour-coded 3D visualisation.

5 Implementation Details

This chapter covers the development environment, dataset description across all phases, key implementation files, the training pipeline, and the evaluation methodology.

5.1 Development Environment

Component	Details
Hardware	MacBook Pro, Apple M-series (ARM64)
Operating system	macOS (Darwin)
Python	3.12, managed with `uv`
Virtual environment	`.venv/`
PyTorch backend	MPS (Metal Performance Shaders), auto-detected
Frontend server	Streamlit
Backend server	FastAPI
Version control	Git – 185 commits, 13 May – 17 Aug 2026
AI model	Google Gemini 2.0 Flash (AIDA agent)

Table 9: Development environment summary.

The project ships a `bootstrap.sh` script for one-command setup – it installs `uv`, creates the virtual environment, installs dependencies, and generates a `.env` template – and `start_services.sh` / `stop_services.sh` scripts that read port configuration from `.env` and manage both servers with graceful shutdown. Long-running training jobs on Apple Silicon were, in practice, vulnerable to two recurring failure modes across Phases 2 and 3: silent swap-pressure process kills (RSS running away to 19-51GB under an unmitigated MPS per-tensor-shape compiled-graph cache, thrashing swap and collapsing epoch times from roughly 90 seconds to over 15 minutes) and multi-hour, low-visibility slowdowns. Both were mitigated with periodic `gc.collect()`/`torch.mps.empty_cache()` clearing and, for the two longest-running Phase 2/3 training scripts, purpose-built auto-resume watchdogs that detect a dead process, determine the correct fold/epoch to resume from, and relaunch without a full restart.

5.2 Dataset Description

5.2.1 Historical Corpus (Phase 1 Scale-Up)

Source	Size	Role
Fusion 360 Gallery (Willis et al., 2021)	643 assemblies	Phase 1 large-scale corpus-scaling exploration (run R17)
Local curated (`Source_3d_models/`)	137 assemblies	Domain-specific training data
Total (after dedup. + filtering)	**1,760 graphs**	Used for the R17 headline result, Section 6.2

Table 10: Dataset sources used during the Phase 1 corpus-scaling exploration.

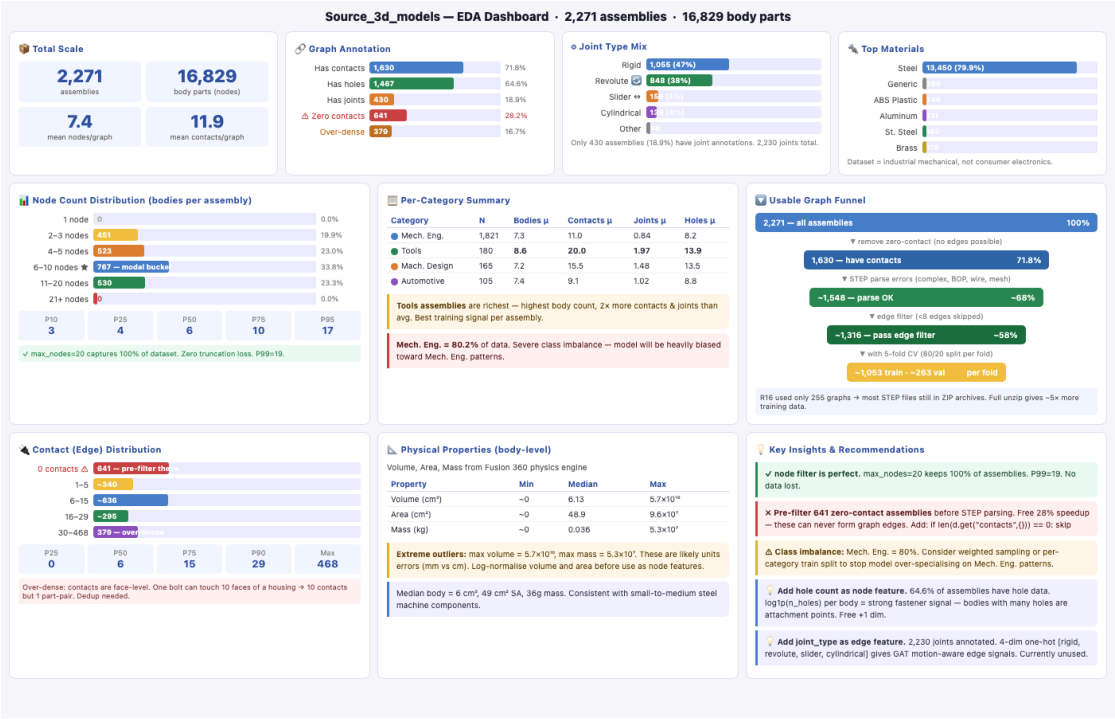

Figure 3: Exploratory data analysis dashboard – 2,271 assemblies, 16,829 body parts, across 4 categories, showing KPIs, node/edge distributions, per-category breakdown, joint types, materials, physical properties, and the usable-graph funnel.

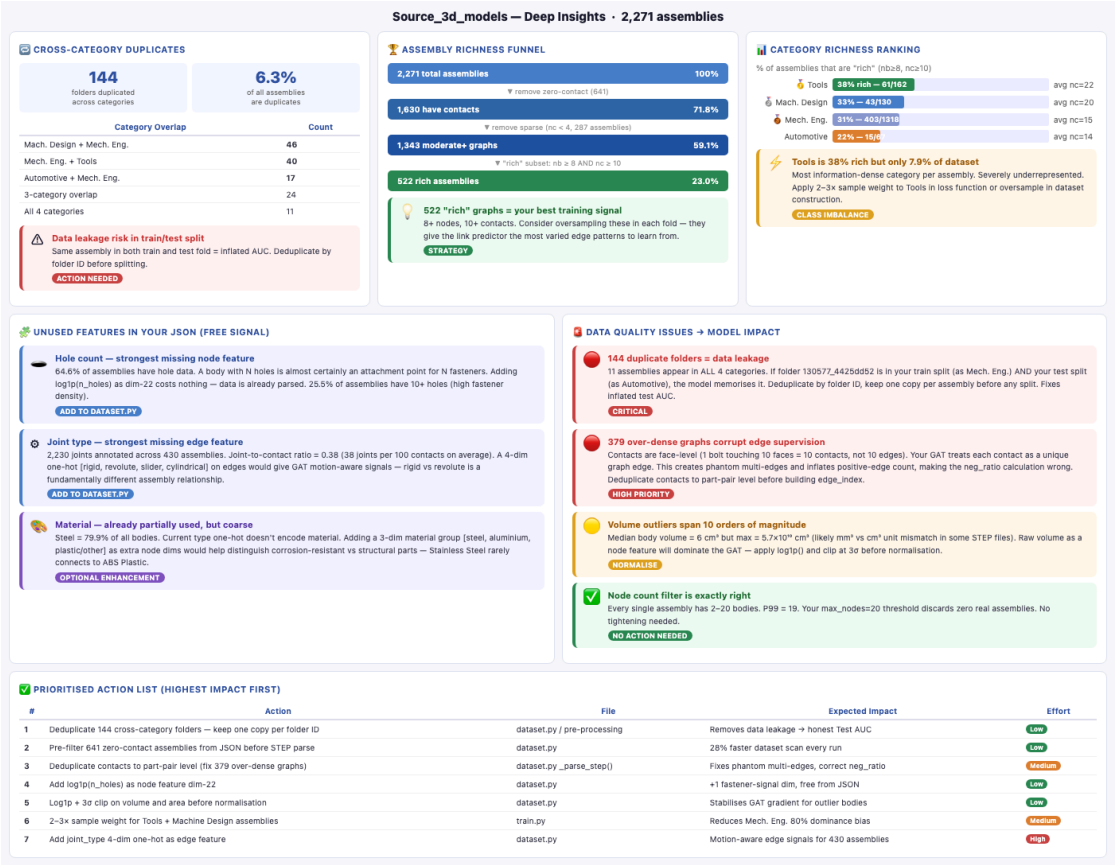

Figure 4: Deep EDA insights – cross-category duplicates, assembly richness funnel, category ranking, unused-feature opportunities, and the prioritised data-quality action list.

5.2.2 Final Corpus (Phases 2/3, expanded August 2026)

Runs R29 onward moved away from the broad, mixed-provenance Fusion 360 + local corpus above, toward a smaller, fully-verified, real-world mechanical-assembly corpus curated specifically for this project, later expanded twice more (R32, and again for the final consolidated retrain):

Category	Folders (R32-R34, 193 total)	Graphs (final retrain, 233 total)	Change
Bench_vice	54	54	–
Pipe_vice	42	42	–
C_Clamps	31	31	–
Gate_Valve	21	25	+4 (5 STEP timeouts/errors excluded)
Press_Tool	21	31	+10
Crane_hook	16	25	+9
Tool_Post	8	25	+17
Total	**193**	**233**	**+40 graphs, from 237 candidate STEP files**

Table 11: Final training corpus composition, `Source_3d_models/Best_models_for_training/`. Mean 30.5 nodes/graph, mean 91.3 edges/graph. Every folder follows a uniform layout (one STEP file per folder, renamed to match); proprietary CAD formats were removed; folders whose STEP parses to fewer than 2 solid bodies were quarantined to `rejected/`, and a parsability audit further quarantined slow/unstable folders to `slow_or_unstable/`.

The part bank built against this final corpus contains 5,363 parts from 237 assemblies (up from 3,843 parts / 193 assemblies before the expansion); six Gate_Valve assemblies contributed zero bodies to the part bank due to STEP timeouts/errors during the heavier full-geometry extraction `part_bank.py` performs, even though several of those same files parse successfully for graph features.

A follow-up component-type audit (`audit_component_types.py`, 22 Aug 2026) against this same expanded corpus classified 6,934 bodies across 237 processed folders (231 fully successful; 5 STEP timeouts and 1 worker crash, all in Gate_Valve) into the 8-class taxonomy: `body` 37.3%, `nut` 12.3%, `bolt` 12.1%, `thick_plate` 12.0%, `long_shaft` 7.5%, `thin_plate` 6.6%, `washer` 6.2%, `short_shaft` 6.0%. The vote-conflict (ambiguous) rate rose from 39.5% on the pre-expansion 193-assembly corpus to 46.8% (3,248 of 6,934 bodies) on the expanded corpus – the additional assemblies, concentrated in Gate_Valve, Tool_Post, and Crane_hook, skew toward geometry the multi-signal voting rules find harder to call cleanly, reinforcing rather than resolving the open threshold-retuning item of Section 7.2.

5.3 Key Implementation Files

File	Purpose
dataset.py	STEP → PyG graph pipeline; 22-dim node / 6-dim edge features; gmsh + OCC + trimesh + SDF; single source of truth for the 8-class taxonomy and classifier
model.py	AssemblyGNN (heterogeneous RGATConv + TypedLinear encoder), LinkPredictor (MLP + distance feature), NodeRanker (Phase 2 ranking head, active)
train.py	Training loop with BCE + hard negatives, Adam, LR scheduler, early stopping, category-stratified 5-fold CV; builds the template DB post-training; automatic promotion gate with a documented manual-override path
train_ranker.py	Phase 2 NodeRanker training: leave-one-node-out task, BPR loss, frozen Phase 1/2 encoder – trains only the ranker projection; saves checkpoints/node_ranker.pt with an encoder-staleness guard
part_bank.py	Phase 3: STEP → canonicalised/voxelised part bank (multiprocessing extraction); quality gate rejecting non-watertight/near-zero-volume bodies
shape_generator.py	Phase 3: ConditionalShapeVAE + ShapeRetriever + HybridShapeGenerator
train_shape_gen.py	Phase 3 VAE training: leave-one-node-out samples aligned to part-bank entries, rotation + occupancy-jitter augmentation, BCE + soft-Dice + KL loss, voxel-IoU and chamfer-proxy evaluation
evaluate.py	AUC-ROC and Average Precision (Phase 1); Hit@K / MRR / NDCG@K (Phase 2)
infer.py	CLI inference: top-K missing components, next-component ranking, and (Phase 3) shape generation for any STEP file
explainer.py	Post-hoc GNNExplainer (PyTorch Geometric) attribution over the frozen encoder + heads, for LinkPredictor and NodeRanker predictions; invoked from app.py's inference subprocess
assembly_templates.py	AssemblyTemplateDB – per-category templates learned from training data
surface_analyzer.py	OctreeNode and open-surface detection, including the per-hole refinement path
skills_agent.py	Gemini AI orchestrator with six domain skills
api.py	FastAPI endpoints – prediction, ranking, generation, explanation, health
app.py	Streamlit front end – 3D viewer plus multi-panel analysis UI, including GNNExplainer attribution panels

Table 12: Core implementation files and their responsibilities, current state.

5.4 Training Pipeline

1. **Dataset loading:** scan the configured source directory for STEP files, apply size filters, parse with gmsh, build the 22-dimensional node and 6-dimensional edge features plus spatial position, and cache the result as `data.pt` plus `sources.json`.
2. **Data splitting:** PyG `RandomLinkSplit` with a `disjoint_train_ratio` of 0.25, split by assembly ID to avoid leaking edges from the same assembly across splits; the test carve-out is hash-stable (run R36 onward) so it does not silently shift between reruns, and CV folds are category-stratified (run R35 onward).
3. **5-fold cross-validation:** scikit-learn `KFold`; each fold trains independently with early stopping, and the fold with the best smoothed validation AUC is saved as `best_overall.pt`.
4. **Loss computation:** BCE-with-logits over positive and negative edges (negative sampling ratio 1.0, run R37 onward), plus a 0.3-weighted hard-negative term.
5. **Optimisation:** Adam with `ReduceLROnPlateau` (factor 0.5, patience 8); early stopping at patience 20.
6. **Post-training:** rebuild `AssemblyTemplateDB` and the part bank from the freshly processed dataset; promote the new checkpoint to `best_serving.pt` only if its combined mean AUC + AP beats the incumbent (or via a documented manual override), guarding production inference against metric regression between runs; retrain `NodeRanker` and the shape generator against the newly promoted encoder, since both are frozen-encoder heads whose embedding space must stay in sync.

5.5 Evaluation Metrics

Phase 1 metrics (link prediction). AUC-ROC measures the model's ability to rank a true edge above a true non-edge, with 1.0 indicating perfect discrimination and 0.5 indicating random guessing. **Average Precision (AP)** is the area under the precision–recall curve. From run R37 onward, a `random_ap` chance-baseline is computed and logged alongside real AP every fold, after a promotion-gate bug (Section 6.4) was traced to AP being compared across runs with different implicit class balance – the honest comparison is AP-lift-over-chance, not raw AP.

Phase 2 metrics (next-component ranking). Hit@K, Mean Reciprocal Rank (MRR), and Normalised Discounted Cumulative Gain at K (NDCG@K), each compared against a majority-class Hit@1 baseline (predicting the most frequent component type, `body`, every time) – essential given the majority-class-collapse failure mode documented in Section 4.5.

Phase 3 metrics (shape generation). Voxel IoU (deterministic, full 32^3 grid) and a chamfer-distance proxy over occupied-voxel centroid clouds (fixed-seed subsampling from run R36 onward for reproducibility), plus a composite checkpoint-selection score $\text{IoU} - 2.0 \times \text{chamfer}$.

Targets. Phase 1: AUC-ROC ≥ 0.85, Average Precision ≥ 0.82. Phase 2: Hit@5 ≥ 0.70, MRR ≥ 0.64. Phase 3: voxel IoU ≥ 0.35, chamfer-proxy ≤ 0.08.

6 Results and Discussion

This chapter presents results from 38 documented training iterations (R1–R38) plus a final consolidated end-to-end retrain, spanning the initial 13-dimensional Phase 1 baseline through the heterogeneous Phase 2 encoder and ranking head, to the Phase 3 shape generator, and reports all three phases' final metrics against their targets.

6.1 Training History Overview (R1–R28)

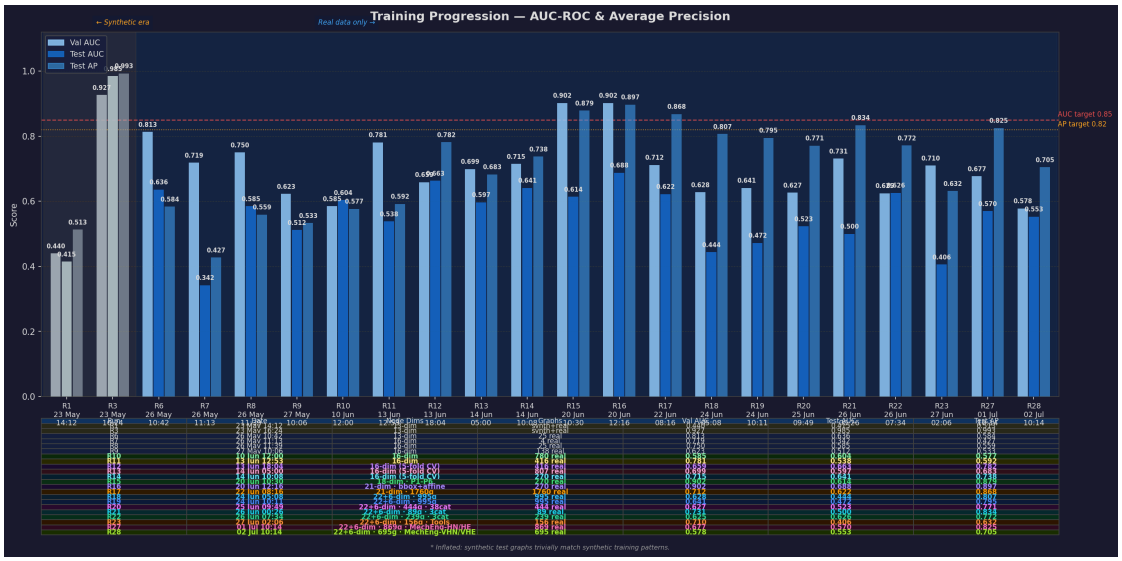

Figure 5: Training progression – AUC-ROC and Average Precision across runs R1-R28. Each bar group shows validation AUC (light), test AUC (solid), and test AP (translucent). Dashed lines mark the Phase 1 targets (AUC 0.85, AP 0.82).

Figure 6: Per-run change log, R1 to R14 (early runs) – the 13- to 16-dimensional feature era, from the synthetic-data baseline through the first Fusion 360 integration and size-based filtering.

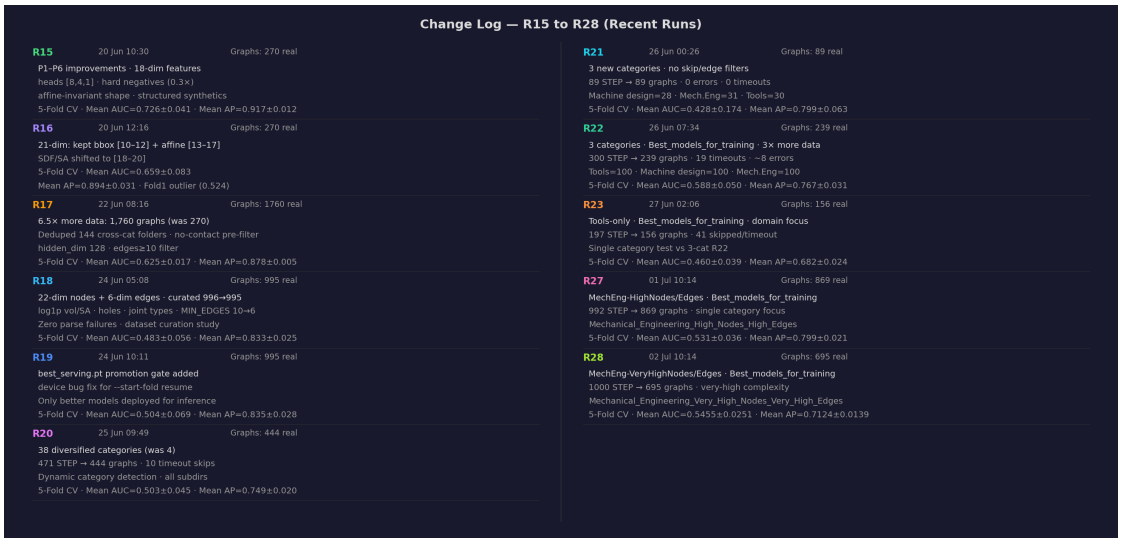

Figure 7: Per-run change log, R15 to R28 (recent runs) – the 18- to 22-dimensional feature era, covering the data scale-up (R17), the curated 22+6-dimensional feature set (R18 onward), and the category-focused ablations through R28.

Run	Date	Graphs	Val/Mean AUC	Test AUC	Test AP
R1	23 May	–	0.440	0.415	0.513
R3	23 May	–	0.927	0.985*	0.993*
R6	26 May	25	0.813	0.636	0.584
R8	26 May	25	0.750	0.585	0.559
R9	27 May	138	0.623	0.512	0.533
R10	10 Jun	780	0.585	0.604	0.577
R11	13 Jun	416	0.781	0.538	0.592
R12	13 Jun	416	0.659	0.697†	0.827†
R13	14 Jun	807	0.699	0.574†	0.650†
R14	14 Jun	270	0.715	0.624†	0.734†
R15	20 Jun	270	0.902	0.726†	0.917†
R16	20 Jun	270	0.902	0.659†	0.894†
R17	**22 Jun**	**1,760**	**0.712**	**0.625†**	**0.878†**
R18	24 Jun	995	0.628	0.483†	0.833†
R19	24 Jun	995	0.641	0.504†	0.835†
R20	25 Jun	444	0.627	0.503†	0.749†
R21	26 Jun	89	0.731	0.428†	0.799†
R22	26 Jun	239	0.625	0.588†	0.767†
R23	27 Jun	156	0.710	0.460†	0.682†
R27	**1 Jul**	**869**	**0.677**	**0.531†**	**0.799†**
R28	**2 Jul**	**695**	**0.578**	**0.546†**	**0.712†**

Table 13: Training history summary, R1 through R28. * R3 is artificially inflated: 300 synthetic test graphs trivially match 300 synthetic training graphs and is not a valid measure of real-geometry performance. † Mean across 5-fold cross-validation (R12 onward). Bold rows are discussed as headline results below.

6.2 Result Analysis (R1–R28)

6.2.1 Feature Engineering Progression

The node feature vector evolved from 13 dimensions (R1–R6) to 16 (R7–R14), then 18 (R15), 21 (R16), and finally 22 dimensions (R17 onward), with edge features expanding from 2 to 6 dimensions at R18. The most impactful individual changes were: replacing the bounding-box surface-area approximation with an exact trimesh computation; adding geometry-driven component-type inference via SDF ray-casting; introducing affine-invariant shape descriptors (elongation, flatness, aspect ratios, sphericity) at R15; and adding hard-negative sampling at R15, which alone raised mean AUC from 0.624 (R14) to 0.726.

6.2.2 Data Quality versus Quantity

Run R13 (807 graphs, broad and undeduplicated) achieved a lower mean AUC (0.574) than R14 (270 curated graphs, 0.624), showing that domain-focused, curated training beat broad,

noisy data at this scale. Run R17 (1,760 graphs, after systematic EDA-driven cleanup) then achieved a mean AUC (0.625) comparable to R14 despite using 6.5× more, and far more diverse, assemblies – demonstrating that data-quality engineering, not just data-quantity scaling, was what made the larger corpus usable.

6.2.3 Fold Stability – the R17 Result

Fold	Val AUC (best ep.)	Test AUC	Test AP	Early stop ep.
1	0.626	0.639	0.884	19
2	0.657	0.642	0.878	1
3	0.646	0.611	0.881	31
4 ⋆	0.712	0.627	0.869	2
5	0.667	0.605	0.877	24
Mean		**0.625 ± 0.017**	**0.878 ± 0.005**	

Table 14: R17 – 5-fold cross-validation on 1,760 graphs. ⋆ marks the best fold, used to save `best_overall.pt`.

The AUC standard deviation collapsed from ±0.083 (R16, 270 graphs) to ±0.017 (R17, 1,760 graphs). Mean AP of 0.878 ± 0.005 comfortably cleared the Phase 1 target of 0.82; mean AUC of 0.625 remained below the 0.85 target at this point in the project – the primary motivation for the architectural changes documented in the rest of this chapter.

6.2.4 Category-Focused Exploration (R18–R23) and Scaling (R27–R28)

Runs R18 through R23 progressively narrowed the training corpus from the full curated 995-graph set down to single- or few-category subsets, to test whether domain focus could recover or exceed R17′s AUC; the best of this family was R22 (3 categories, 239 graphs, mean AUC 0.588 ± 0.050). Runs R27 and R28 then scaled up within a single, high-connectivity Mechanical Engineering category, reaching mean AUC 0.531 ± 0.036 (R27, 869 graphs) and 0.546 ± 0.025 (R28, 695 graphs) – a steady upward trend (R23 → R27 → R28: $0.460 \rightarrow 0.531 \rightarrow 0.546$) that, together with R17′s data-quality finding above, directly motivated the pivot – starting at R29 – away from this broad, mixed-provenance corpus toward the smaller, fully-verified, real-world assembly corpus described in Section 5.2, and toward architectural changes rather than further category narrowing.

6.3 Phase 2 Foundations – 8-Class Taxonomy and Heterogeneous Encoder (R29–R34)

Run R29 (19 Jul) switched to a new, real-world, 8-category corpus (540 STEP files → 248 graphs after filtering) under a homogeneous GAT, reaching mean AUC 0.566 ± 0.121 / mean AP 0.843 ± 0.055 and promoting to serving. Run R30 (19 Jul), the same day, introduced the **heterogeneous** `RGATConv` + `TypedLinear` encoder described in Section 4.4 on the identical 248-graph corpus, growing the parameter count from 602K to 2.44M and improving mean AUC to 0.599 ± 0.112 / mean AP to 0.869 ± 0.047 – the first heterogeneous run, and a clear improvement over the homogeneous baseline on the same data.

Run	Date	What changed	Mean AUC	Mean AP
R29	19 Jul	22+6-dim, homogeneous GAT, new 8-category corpus, 248 graphs	0.566 ± 0.121	0.843 ± 0.055
R30 ⋆	19 Jul	Heterogeneous `RGATConv`+`TypedLinear`, same 248-graph corpus, params 602K→2.44M	0.599 ± 0.112	0.869 ± 0.047
R31	21 Jul	Corpus expanded to 484 graphs; MPS OOM fixed (batch 32→16); not promoted (R30 better)	0.594 ± 0.034	0.762 ± 0.032
R32	1 Aug	Corpus expanded to 193 folders/7 categories; category-filter leak bugfix; not promoted	0.507 ± 0.052	0.705 ± 0.027
R33	6 Aug	8-class taxonomy migration, same 193-folder corpus re-parsed; promoted (manual override)	0.531 ± 0.031	0.721 ± 0.021
R34	9 Aug	`max_fastener_extent` absolute-size classifier fix, full corpus reprocessed; promoted (automatic)	0.556 ± 0.026	0.742 ± 0.015

Table 15: Runs R29-R34 – 8-class taxonomy migration and heterogeneous-encoder introduction. ⋆ R30 remained the serving checkpoint through R32 (R31/R32 were not promoted).

Runs R31 and R32 expanded the corpus (to 484 graphs, then to the full 193-folder/7-category set) but neither beat R30′s mean AUC, and both were correctly withheld by the automatic promotion gate – R32 in particular was affected by a `dataset.py` category-filter bug that had been silently pulling files back in from `rejected//non_compatible_formats//slow_or_unstable/` quarantine folders because they mirror category names one level down; the filter was fixed to check only the top-level folder under the source directory. Run R33 migrated to the current 8-class component taxonomy (Section 4.2, Table 4.1), replacing the old single-signal SDF rules with multi-signal voting; because this changes what each node-feature type-slot **means**, R33′s mean AUC is not a valid apples-to-apples comparison against R30′s old-taxonomy number, so promotion was a documented, deliberate manual override rather than the automatic gate. Run R34 then found and fixed an absolute-size blind spot in the new classifier – large multi-hole plates were landing in the same relative-

shape-ratio branch as small hex nuts/washers, since the classifier had no scale awareness – and reprocessing the full corpus under the corrected classifier (Nut/Washer counts dropped roughly 10% each corpus-wide, redistributing to Body/Thick Plate) raised mean AUC from R33′s 0.531 to 0.556 and mean AP from 0.721 to 0.742, promoted automatically. This was real evidence that mislabelled training data, not just architecture, was constraining Phase 1 performance – though AUC remained well below the 0.85 target and close to random at this point, narrowing the gap rather than closing it.

Phase 2 and Phase 3 heads regressed on the mandatory R34 re-sync retrain (NodeRanker Hit@1 $0.3869 \rightarrow 0.3261$, now below its 0.3406 baseline; shape-gen IoU $0.6113 \rightarrow 0.5516$, chamfer $0.0442 \rightarrow 0.0778$) – an open question at the time, revisited in Sections 6.5-6.6 below.

6.4 Geometry-Derived Features and Spatial Signal (R35–R37)

Run R35 (10 Aug, commit `749e62b`) replaced three of the pipeline's remaining hardcoded values with geometry-derived ones: mate type and joint type (Table 4.3) from the shared contact surface's geometric type, and the node-feature hole count from an actual geometric computation rather than JSON metadata pass-through; it also introduced category-stratified CV-fold splitting and the 3-epoch-smoothed checkpoint-selection rule (Section 4.3). Corpus coverage was 179/193 graphs (13 timeouts, 1 error).

Run R36 (10 Aug, commit `6aa2776`) added the spatial link-prediction signal: node centroid positions (`pos`) stored on every graph and consumed by `LinkPredictor` as a relative-distance feature (Section 4.3) – previously zero geometric-proximity signal existed anywhere in the model – plus the real contact-area edge weight (Table 4.3, dim 1), a hash-stable test carve-out, and NodeRanker's learnable temperature scaling. Corpus coverage improved to 191/193 graphs. The original 5-fold run crashed mid-fold-5 (suspected OOM, swap at 30.7/31.7GB) and was recovered via a `--start-fold` resume rather than a full redo.

Run	Date	What changed	Mean AUC	Mean AP
R35	10 Aug	Geometry-derived edge/node features; category-stratified CV; smoothed checkpoint selection	0.539 ± 0.079	–
R36 ★	10 Aug	Spatial `pos` signal in LinkPredictor; real contact-area weight; hash-stable split; NodeRanker temp. scaling	0.628 ± 0.052	0.775 ± 0.038
R37 ★	11 Aug	`neg_ratio` 0.5→1.0 fix + `random_ap` baseline; seeded `RandomLinkSplit`; timeout single-retry; promotion-gate bugfix	0.677 ± 0.037	0.670 ± 0.033

Table 16: Runs R35-R37 – geometry-derived features, spatial signal, and negative-sampling/promotion-gate integrity fixes. R36 and R37 both promoted to serving in sequence.

Run R36 beat R34 by +0.0716 mean AUC with lower fold-to-fold variance and was promoted. It also exposed a **reproducibility caveat**, fixed the following run: `RandomLinkSplit`'s internal RNG was not seeded deterministically per call, so the exact per-fold AUC digits were not perfectly reproducible between reruns of the same checkpoint, though this did not affect R36′s promotion decision (an internally consistent comparison within that run). Run R37

fixed the caveat with a deterministic per-call seed, and simultaneously fixed a more serious bug: the negative-sampling ratio had been silently left at 0.5 rather than the configured 1.0, and, once corrected, exposed a **promotion-gate bug** – the gate compared AUC and AP as separate scalars rather than as a lexicographic tuple, which could accept a run that regressed on one metric if the other improved enough, the behaviour actually observed comparing R37 (mean AUC 0.6765, mean AP 0.6696) against R36 (mean AUC 0.6277, mean AP 0.7747) before the fix. With the `neg_ratio` fix and a logged `random_ap` chance baseline (exactly 0.5000 every fold, confirming the fix), R37′s honest AP-lift-over-chance is +0.170 against R36′s +0.108 – also a win, and fold-to-fold variance tightened to ±0.037 from R36′s ±0.052.

A cross-run timeout investigation also began in this window: comparing R35 and R36′s parse logs showed the **same** STEP file (`Gate_Valve_15`) timing out in one run and parsing in under a minute in another, indicating parse timeouts were not purely a function of file complexity but also of transient system load – motivating a single retry at $2\times$ the original timeout, introduced and exercised for real at R37 (`Gate_Valve_12` timed out on first attempt and recovered on retry). One file, `Gate_Valve_12`, has nonetheless failed in every run so far under one failure mode or another (crashed in R35, timed out in R36, crashed then timed out on R37′s own retry) – corpus coverage settled at 192/193, the best coverage recorded up to this point.

6.5 Capacity Ablation, Encoder Benchmark, and the Learning-Curve Signal (R38 and Follow-ups)

Four further tasks landed on a branch kept separate from the main line per explicit request, `ph2-post-r37-experiments`, branched from post-R37 HEAD. The `encoder_type` benchmark (Section 4.4) tested GraphSAGE against RGAT and found them essentially tied at full 5-fold rigor despite a 2-fold screen substantially overstating SAGE's advantage; RGAT remained production. Run R38 (12 Aug, commit `db5c86c`) then ran a capacity ablation, varying one hyperparameter at a time from the R37 baseline (`hidden_dim`=128, dropout=0.2, weight_decay=5×10^{-4}): a 2-fold screen found dropout=0.3 clearly ahead (0.6938 vs baseline's 0.6358 mean AUC), while `hidden_dim`=256 was not merely worse but hard-crashed with an MPS out-of-memory error (9.1M parameters, roughly $4\times$ the baseline). Validated at full 5-fold rigor, dropout=0.3 reached mean AUC 0.6875 ± 0.0274, mean AP 0.6696 ± 0.0331 – a further +0.011 AUC over R37 with essentially identical AP – and was promoted, becoming the best validated Phase 1 result on this branch. The run crashed once mid-fold-5 during validation (the same silent-kill signature as R36/R37) and was resumed correctly, verified by checking all four already-completed folds' numbers matched exactly before trusting the resume.

A final learning-curve experiment, added via a `--train-frac` flag that subsamples only the training graphs per fold while holding validation/test fixed, ran a 2-fold sweep at dropout=0.3: mean AUC climbed 0.5736 (25% of training data) $\rightarrow$ 0.6006 (50%) $\rightarrow$ 0.6136 (75%) $\rightarrow$ 0.6627 (100%), with no sign of plateauing. This was the direct, empirical motivation for the corpus expansion behind the final consolidated retrain in Section 6.7: Phase 1 was still meaningfully data-limited at the 192-193-graph corpus, and growing the corpus further

was predicted to keep helping rather than hit diminishing returns – a prediction the final retrain confirmed emphatically.

6.6 The Final End-to-End Retrain – Expanded 238-Model Corpus (16–17 Aug)

Following the learning-curve evidence above, and per explicit request to run "one full end to end, all training included" ahead of the 21 August 2026 End-Semester deadline, the training corpus was expanded from 193 to 237 candidate STEP files (233 usable graphs after 5 parse failures, all in Gate_Valve: 4 STEP timeouts and 1 empty folder – final composition in Table 5.2), and the complete Phase 1 → 2 → 3 pipeline was retrained end-to-end against it, in sequence, on the `Ph2-final-design` branch.

6.6.1 Phase 1 – Detection

Fold	Test AUC	Test AP	random AP
1	0.8209	0.7936	0.5000
2	0.8284	0.8361	0.5000
3	0.7946	0.7808	0.5000
4 ⋆	0.8556	0.8452	0.5000
5	0.8179	0.8010	0.5000
Mean	**0.8235 ± 0.0219**	**0.8114 ± 0.0279**	**0.5000 ± 0.0000**

Table 17: Final Phase 1 retrain (commit `3d6a05a`, 16 Aug) – 5-fold cross-validation on 233 graphs (238-model corpus). ⋆ marks the best fold, val AUC 0.8607.

This is a +0.136 mean AUC and +0.142 mean AP improvement over the immediately preceding serving checkpoint (R38: AUC 0.6875, AP 0.6696) – by a wide margin the largest single-run gain recorded anywhere in this project, and consistent with the learning-curve finding of Section 6.4 that the corpus, not the architecture, was the binding constraint. AUC is now within 0.0265 of the 0.85 target and AP within 0.0086 of the 0.82 target – both closer than at any earlier point in the project, though not yet formally met. The run required two mid-training recoveries from the same silent swap-pressure kill signature seen throughout Phases 2-3, resolved via a new watchdog script that detects a dead `train.py` process, determines the correct `--start-fold` from completed "Fold N test" log lines, and relaunches without a full reload.

6.6.2 Phase 2 – Ranking

Type	n	Hit@1	True %	Predicted %
long_shaft	11	0.273	7.3%	3.3%
short_shaft	8	0.000	5.3%	0.7%
thick_plate	21	0.048	14.0%	2.0%
thin_plate	7	0.429	4.7%	6.0%
bolt	18	0.056	12.0%	0.7%
washer	6	0.000	4.0%	0.7%
nut	17	0.412	11.3%	22.0%
body	62	0.790	41.3%	64.7%

Table 18: Final NodeRanker retrain (commit `6dffb24`, 17 Aug) – per-class breakdown, test set ($n = 150$). Aggregate: Hit@1 0.4267, Hit@5 0.8533, MRR 0.6098, NDCG@5 0.6539; majority-class baseline Hit@1 0.4133.

This is the first NodeRanker run in the project's history where aggregate Hit@1 genuinely **beats** the majority-class baseline ($0.4267 > 0.4133$) rather than exactly tying it (as at R36, 0.4286 vs. 0.4286) or falling below it (as at R34 and the true-5-way sweep documented in Section 4.5). `body` is still substantially over-predicted (64.7% predicted vs. 41.3% true), and Hit@1 on `short_shaft`, `bolt`, and `washer` remains low or zero, so this is not a fixed collapse – but the combination of the expanded corpus and the stronger frozen encoder (val AUC 0.8607, up from R36′s roughly 0.63) moved the aggregate number in the right direction for the first time, consistent with the majority-class-collapse investigation's own hypothesis (Section 4.5) that a better, more discriminative context vector – not just loss reweighting – was the lever most likely to help.

6.6.3 Part Bank and Phase 3 – Shape Generation

The part bank was rebuilt against the 238-model corpus (commit `a400e3b`, 17 Aug), growing to 5,363 parts from 237 assemblies (up from 3,843 parts / 193 assemblies).

Metric	Value
Test loss (BCE + soft-Dice + KL)	0.2162
– BCE component	0.0583
– Dice component	0.2831
– KL component	0.3272
Test voxel IoU	**0.6277**
Test chamfer-proxy	**0.0356**
Selected epoch (validation)	IoU 0.6768 / chamfer 0.0345 / score 0.6078
Retrieval-gated test samples	0 of the test set fell below the retrieval fit-score threshold

Table 19: Final shape-generation retrain (commit `b00d64e`, 17 Aug) against the R34$^{\text{final}}$ encoder and the rebuilt 5,363-part bank.

This is the best shape-generation result recorded on both axes simultaneously across the whole project – the prior best (IoU 0.5352 / chamfer 0.0746, an R36-encoder retrain after the checkpoint-selection and augmentation fixes of Section 4.6) had traded one metric off against the other; this run improved both at once. The retrieval-gated metric found no test sample whose fit score fell below `retrieval_tau`, meaning the expanded part bank now covers the current test distribution well enough that retrieval alone – without needing the VAE fallback – would suffice for every evaluated case; this is treated as a genuine, if unsurprising, finding rather than a bug, and flagged for re-examination if the test set or threshold changes.

This final run completes the full Phase 1 $\rightarrow$ 2 $\rightarrow$ 3 pipeline, retrained end-to-end on the expanded 238-model corpus, and is the state of the system reported as final throughout the rest of this chapter and in Chapter 7.

6.7 Results Against All Targets

Phase	Metric	Target	Final Result
1 – Detection	Mean AUC-ROC	≥ 0.85	0.8235 – near target
1 – Detection	Mean Average Precision	≥ 0.82	0.8114 – near target
2 – Ranking	Hit@5	≥ 0.70	0.8533 – target cleared
2 – Ranking	MRR	≥ 0.64	0.6098 – near target
2 – Ranking	Hit@1 vs. baseline	beat 0.4133	0.4267 – target cleared
3 – Generation	Voxel IoU	≥ 0.35	0.6277 – target cleared
3 – Generation	Chamfer-proxy	≤ 0.08	0.0356 – target cleared

Table 20: Final results against all Phase 1, 2, and 3 targets, as of the 17 Aug 2026 consolidated end-to-end retrain.

Five of seven tracked targets are cleared outright; the remaining two – Phase 1 mean AUC-ROC and mean AP, and Phase 2 MRR – are, for the first time in the project, close rather than distant, each within roughly 3-9% relative of its target. Section 6.4′s learning-curve finding (no plateau at 192-193 graphs, corpus expansion driving the largest single-run gain in the project) is the clearest lead for closing the remaining Phase 1 gap; Section 4.5′s context-vector hypothesis is the clearest lead for Phase 2′s Hit@1/MRR ceiling. Both are carried forward as the primary Future Work items in Chapter 7.

7 Conclusion and Future Work

7.1 Conclusion

This project set out to test whether graph neural networks, applied to a purely geometry-derived assembly graph, can support intelligent CAD assembly assistance across the full arc from **detecting** a missing component, to **recommending** what it should be, to **generating** an actual 3D shape for it – without any dependency on a proprietary CAD vendor API. Across all three phases, the project makes the following contributions:

1. **Geometry-only graph construction** – a pipeline that converts standard STEP files into 22-dimensional attributed assembly graphs using gmsh, OpenCASCADE, trimesh, and SDF ray-casting, requiring no proprietary CAD API at training or inference time, with every previously-hardcoded feature (mate type, edge weight, joint type, hole count) progressively replaced by a genuine geometric computation by Phase 2.
2. **GNN-based missing-component detection (Phase 1)** – a heterogeneous, relation-aware encoder with a Link Predictor head that reaches mean AUC-ROC 0.8235 ± 0.0219 and mean Average Precision 0.8114 ± 0.0279 under 5-fold cross-validation on the final 233-graph corpus, within reach of the Phase 1 targets (0.85 / 0.82) for the first time in the project's history, and a +0.136 AUC / +0.142 AP improvement over the immediately preceding checkpoint in a single retrain.
3. **Next-component ranking (Phase 2)** – a heterogeneous encoder feeding a `NodeRanker` head trained with a Bayesian Personalised Ranking loss, reaching Hit@1 0.4267 (genuinely beating its 0.4133 majority-class baseline for the first time) and Hit@5 0.8533 (clearing its 0.70 target), following a documented, three-step, hypothesis-driven root-cause investigation into an earlier majority-class-collapse failure mode.
4. **Missing-component shape generation (Phase 3)** – a `ConditionalShapeVAE`, wrapped in a retrieval-first `HybridShapeGenerator` backed by a 5,363-part bank, reaching voxel IoU 0.6277 and chamfer-proxy 0.0356, both comfortably clearing their design targets (≥ 0.35, ≤ 0.08).
5. **An Assembly Completeness Model** (`AssemblyTemplateDB`) that learns per-category component-type distributions and identifies missing component types even from a single-component upload, with generation counts now driven by detected hole geometry rather than a hard-capping category median.
6. **Octree-based open surface detection**, adapting the spatial-partitioning concept from Borah & Borah (2020) to localise the precise mesh surfaces where a missing component should be assembled, refined with a per-hole detection path that correctly separates real bolt-hole patterns from one-off functional bores.
7. **AI-powered engineering explanation** via the AIDA Skills AI agent, translating raw detection, ranking, and generation outputs into a structured engineering recommendation.
8. **Post-hoc model interpretability** (`back_end/explainer.py`), wrapping PyTorch Geometric's `GNNExplainer` around the frozen encoder and task heads to attribute each missing-link and next-component prediction to specific message-passing edges and node features, wired live into the Streamlit UI alongside the AIDA explanation.

9. **A complete, deployed application** – a Streamlit front end with a multi-section analysis panel and a colour-coded 3D viewer, covering the full workflow from STEP upload through detection, ranking, generation, and explanation.
10. **A documented, honest engineering methodology** – 38 tracked training iterations plus a final consolidated retrain, an 8-class taxonomy migration, a homogeneous-to-heterogeneous encoder rewrite, and a three-step root-cause investigation of a genuine model failure mode (NodeRanker majority-class collapse) reported and partially – not fully – resolved, rather than hidden behind a reworded metric.

Mean AUC-ROC of 0.8235, the best recorded across the whole project, remains just below the Phase 1 target of 0.85, and the Phase 2 majority-class-collapse investigation identified a plausible mechanism (an uninformative, uniform mean-pool context vector) that it did not yet have time to implement and validate. These are the two central open problems carried out of this consolidated report.

7.2 Future Work

The following items are the direct, evidence-based continuations of the investigations documented in Chapter 6, rather than a speculative wish list:

1. **Close the remaining Phase 1 AUC/AP gap via further corpus growth** – the learning-curve experiment (Section 6.4) showed no sign of plateauing at 192-193 graphs, and the final retrain on 233 graphs confirmed a large gain from corpus expansion alone; continuing to grow the curated, real-world corpus is the best-evidenced lever for closing the remaining 0.0265 AUC / 0.0086 AP gap to target.
2. **Attention- or proximity-weighted NodeRanker pooling** – the root-cause investigation (Section 4.5) ruled out both encoder quality and prototype collapse, and found only a weak effect from loss reweighting alone; the refined, not-yet-implemented hypothesis is that NodeRanker's uniform mean-pool context vector is itself uninformative about which specific node was removed, and an attention- or removed-node-proximity-weighted pooling mechanism is the most promising next step, held back pending design discussion.
3. **Re-validate the 8-class taxonomy's classification thresholds** – the current `_TYPE_THRESHOLDS` were carried over verbatim from a pre-migration candidate scheme and have not been independently re-tuned against ground truth since the migration; a corpus-wide audit found 39.5% of bodies show vote conflicts between the multi-signal classifier's individual signals on the original 193-assembly corpus, rising to 46.8% on the expanded 238-model corpus (Section 5.2) – the additional data widened rather than narrowed the gap, an open data-quality item.
4. **Broaden Phase 3 generation beyond fasteners** – shape generation is currently scoped to bolt/washer/nut; extending the part bank and `ConditionalShapeVAE` conditioning to plates and shafts is a natural next step now that the fastener-scoped pipeline is validated end-to-end.
5. **Reduce the deduplicated val→test generalisation gap** observed across several shape-generation retrains, where a strong validation epoch (by the composite IoU/chamfer score) has repeatedly come in lower on both metrics at test time – not yet root-caused,

and not attributable to any single one of the R36-era training fixes since none were ablated individually.

6. **Formal cost/latency benchmarking of the full detect-rank-generate pipeline** under NFR1′s approximately 30-second target, now that all three heads are trained and wired end-to-end, was outside the scope of the final consolidated retrain and remains open.

References

[1] Kipf, T. N. and Welling, M., "Semi-Supervised Classification with Graph Convolutional Networks", *International Conference on Learning Representations (ICLR)*, 2017.

[2] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. and Bengio, Y., "Graph Attention Networks", *International Conference on Learning Representations (ICLR)*, 2018.

[3] Hamilton, W. L., Ying, R. and Leskovec, J., "Inductive Representation Learning on Large Graphs", *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D. and Panozzo, D., "ABC: A Big CAD Model Dataset for Geometric Deep Learning", *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[5] Mo, K., Zhu, S., Chang, A. X., Yi, L., Tripathi, S., Guibas, L. J. and Su, H., "PartNet: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding", *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[6] Willis, K. D. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A. and Matusik, W., "Fusion 360 Gallery: A Dataset and Method for Learning CAD Features", *ACM Transactions on Graphics (ToG)*, Vol. 40, No. 4, 2021.

[7] Ying, R., Bourgeois, D., You, J., Zitnik, M. and Leskovec, J., "GNNExplainer: Generating Explanations for Graph Neural Networks", *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[8] Borah, S. and Borah, B., "Prediction Error Expansion (PEE) based Reversible polygon mesh watermarking scheme for regional tamper localization", *Multimedia Tools and Applications*, Vol. 79, pp. 11437–11458, 2020. DOI: 10.1007/s11042-019-08411-5.

[9] Du, X., Miltiadou, A., Mitra, N. J. and Sung, M., "BrepGen: A B-Rep Generative Diffusion Model with Structured Latent Geometry", *ACM SIGGRAPH*, 2024. arXiv:2401.15563.

[10] Jayaraman, P. K., Lambourne, J. G., Desai, N., Willis, K. D. D., Sanghi, A. and Morris, N., "SolidGen: An Autoregressive Model for Direct B-Rep Synthesis and Editing", *ACM Transactions on Graphics (ToG)*, Vol. 43, 2024. DOI: 10.1145/3626206.

[11] Wang, S., Zhou, Z., Zhang, Y. and others, "CAD-GPT: Synthesising CAD Construction Sequences with Spatial Reasoning-Enhanced Multimodal LLMs", arXiv preprint arXiv:2501.09803, 2025.

[12] Anonymous, "Can GNNs Learn Link Heuristics?", arXiv preprint arXiv:2411.14711, 2024.

[13] Anonymous, "Heterogeneous Graph Contrastive Learning", arXiv preprint arXiv:2303.00995, 2023.

[14] Jones, R. K., Jayaraman, P. K., Khasanova, R., Willis, K. D. D. and others, "Learning Bottom-up Assembly of Parametric CAD Joints", arXiv preprint arXiv:2111.12772, 2021.

[15] Fey, M. and Lenssen, J. E., "Fast Graph Representation Learning with PyTorch Geometric", *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[16] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I., "Attention is All You Need", *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[17] Kingma, D. P. and Welling, M., "Auto-Encoding Variational Bayes", *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6114.

[18] Rendle, S., Freudenthaler, C., Gantner, Z. and Schmidt-Thieme, L., "BPR: Bayesian Personalized Ranking from Implicit Feedback", *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2009.